



UNIVERSITY OF PADOVA

DEPARTMENT OF MATHEMATICS

MASTER THESIS IN DATA SCIENCE

HIERARCHICAL BIDIRECTIONAL TRANSFORMERS FOR TEXT SUMMARIZATION

SUPERVISOR

PROF. TOMASO ERSEGHE
UNIVERSITY OF PADOVA

MASTER CANDIDATE

PARISA FOROOZANDE NEJAD

ACADEMIC YEAR

2024-2025

چون نیست ز هرچه هست جز باد به دست
چون هست به هرچه هست نقصان و شکست
انکار که هرچه هست در عالم نیست
پندار که هرچه نیست در عالم هست
خیام

"SINCE FROM ALL THAT IS, NOTHING REMAINS BUT WIND IN THE HAND,
AND SINCE FOR ALL THAT IS, THERE IS FLAW AND COLLAPSE,

ASSUME THAT ALL THAT IS IN THE WORLD, IS NOT;
IMAGINE THAT ALL THAT IS NOT IN THE WORLD, IS." –OMAR KHAYYAM

Abstract

This thesis presents a novel approach to unsupervised extractive text summarization using hierarchical transformer architectures. With the exponential growth of digital content, the need for efficient text summarization methods has become increasingly critical. Our research addresses this challenge by combining the hierarchical bidirectional transformer architecture of HIBERT with the unsupervised ranking criteria of STAS, enhanced by a Pointwise Mutual Information (PMI)-based redundancy control mechanism. The proposed method employs a two-level processing structure that captures both local sentence semantics and global document structure. Unlike traditional approaches that rely on surface-level features, our hierarchical transformer architecture enables effective sentence-level attention mechanisms for ranking sentences in unsupervised extractive summarization. The model processes documents by generating contextual sentence representations through HIBERT’s pre-trained hierarchical encoder, then applies STAS ranking criteria combined with PMI-based redundancy measures to select the most salient sentences. We evaluate our approach on the CNN/DailyMail dataset, a standard benchmark for summarization tasks. The experimental results demonstrate that our method achieves highly competitive performance, with significant improvements attributed to the redundancy control mechanism.

Contents

ABSTRACT	v
LIST OF FIGURES	xi
LIST OF TABLES	xiii
LISTING OF ACRONYMS	xv
ACKNOWLEDGMENTS	i
1 INTRODUCTION	3
1.1 Problem Statement and Motivation	3
1.2 Goal	5
1.3 Thesis Structure Overview	5
2 THEORETICAL BACKGROUND	7
2.1 Fundamentals of Text Summarization	7
2.1.1 Definition and Motivation of Automatic Text Summarization (ATS)	7
2.1.2 Extractive vs. Abstractive Summarization Paradigms	8
2.1.3 Classification of ATS	8
2.1.4 Evaluation Metrics for Summarization	11
2.1.4.1 ROUGE (Recall-Oriented Understudy for Gisting Evalu-	
ation)	11
2.1.4.1.1 ROUGE-N	11
2.1.4.1.2 ROUGE-L	12
2.1.4.1.3 ROUGE-S	12
2.1.4.2 BLEU (Bilingual Evaluation Understudy)	13
2.1.4.3 BERTScore	13
2.1.4.4 Human Evaluation	14
2.1.4.5 Comparative Analysis of Evaluation Metrics	14
2.1.5 Challenges in Summarization	14
2.2 Statistical Methods	15
2.2.1 Term Frequency-Inverse Document Frequency (TF-IDF)	16
2.2.2 Maximal Marginal Relevance (MMR)	16
2.2.3 Graph-Based Methods	17

2.2.3.1	TextRank	17
2.2.3.2	LexRank	17
2.2.4	Limitations of Classical Statistical Methods	18
2.3	Deep Learning Methods	19
2.3.1	Recurrent Neural Networks for Summarization	19
2.3.2	Persistent Context Limitations	20
2.4	Pre-trained Language Models	21
2.4.1	The Transformer Architecture	21
2.4.2	The Self-Attention Mechanism	21
2.4.3	Multi-Head Attention	22
2.4.4	Encoder-Decoder Structure	23
2.4.5	Positional Encoding	24
2.4.6	Input Representation and Embeddings	24
2.4.7	Training Objectives and Pre-training	25
2.4.8	Advantages Over RNNs	25
2.5	Large Language Models	26
2.6	Hierarchical Architectures for Extractive Summarization	26
2.6.1	Problem Formulation	27
2.6.2	Base Models	28
3	RELATED WORKS	29
3.1	Evolution of Text Summarization Paradigms	29
3.1.1	Statistical and Graph-Based Approaches	29
3.1.2	Deep Learning Era	30
3.1.3	The Pre-trained Language Model Era	31
3.1.3.1	BERT and Its Impact	32
3.1.3.2	Adapting PLMs for Summarization	32
3.2	Unsupervised Extractive Summarization with Pre-trained Models	32
3.2.1	Graph-Based Methods Enhanced with Pre-trained Embeddings	33
3.2.1.1	PACSUM	33
3.2.1.2	GUSUM	33
3.2.2	Information-Theoretic Approaches	34
3.2.2.1	PMI-Based Ranking	34
3.2.3	Multi-Modal and Ensemble Approaches	34
3.2.3.1	RANKSUM	34
3.3	Hierarchical Transformer Architectures	34
3.3.1	HIBERT: Hierarchical Bidirectional Encoder Representations from Transformers	35
3.3.2	STAS: Sentence-Level Transformer based Attentive Summarization	35
3.3.3	HiStruct+: Hierarchical Structure-Aware Extractive Summarization	35
3.4	Comparative Analysis of ATS Paradigms	36

4	METHODOLOGY	39
4.1	Extractive Summarization Architecture	39
4.1.1	Input Document(s)	39
4.1.2	Text Pre-processing	40
4.1.3	Feature Extraction	40
4.1.4	Sentence Scoring	40
4.1.5	Base Model	40
4.1.6	Sentence Selection	41
4.1.7	Output Summary	41
4.1.8	Post Processing	41
4.1.9	Final Summary	41
4.2	Introduction to the Proposed Method	41
4.3	Base Models Architecture	42
4.3.1	HIBERT Model	42
4.3.2	STAS Model	44
4.3.2.1	STAS Ranking Criteria	45
4.3.3	Relevance and Redundancy Ranking Criteria	46
4.3.3.1	Relevance Criterion	46
4.3.3.2	Implementation Considerations	47
4.3.3.3	Redundancy Criterion	47
4.3.3.4	Sentence Extraction	48
4.4	Proposed Combined Architecture	48
4.4.1	Combined Ranking Strategy	48
4.4.2	Sentence Selection Algorithm	49
4.4.3	Hyperparameter Tuning	49
4.5	Technical Implementation	52
4.5.1	HIBERT Architecture	52
4.5.2	FAIRSEQ Library	52
4.5.3	Preprocessing Pipeline	53
4.6	Oracle Extraction	54
4.7	Evaluation Metrics	55
5	EXPERIMENTAL RESULTS	57
5.1	Dataset Description	57
5.2	Experimental Setup	58
5.2.1	Hardware Configuration	58
5.2.2	Model Configuration	58
5.2.3	Inference Parameters	59
5.3	Baseline Comparisons	60
5.3.1	Oracle Summaries	60
5.3.2	State-of-the-Art Models	60

5.4	Main Results	61
5.5	Discussion	65
6	CONCLUSION	69
6.1	Main Findings	69
6.2	Limitations	70
6.3	Future Work	70
	REFERENCES	73

Listing of figures

1.1	The Evolution of the Four Major Paradigms in Text Summarization [1] . . .	4
2.1	Model Architecture of An Automatic Text Summarizer	8
2.2	Classifications of the ATS systems	9
2.3	Automatic text summarization approaches and their associated methods.[2] .	11
2.4	Recurrent Neural Network Architecture	20
2.5	The Transformer Architecture	22
2.6	Scaled Dot-Product Attention and Multi-Head Attention	23
2.7	Hierarchical Architecture Pipeline for Extractive Summarization	27
4.1	Architecture of an Extractive Text Summarization System.[1]	39
4.2	HIBERT's Architecture During Training	43
4.3	HIBERT's Extractive Summarization	45
4.4	Architecture of the Proposed Combined Summarization Model	50
4.5	Preprocessing Pipeline Applied to the CNN/DailyMail Dataset.	53

Listing of tables

2.1	Comprehensive Comparison of Automatic Evaluation Metrics for Extractive Text Summarization[1]	15
3.2	Comparative Analysis of Text Summarization Paradigms	37
3.1	Comprehensive Overview of Text Summarization Methods	38
4.1	HIBERT _M architecture specifications	52
5.1	Mean Token Count by Feature in CNN/DailyMail Dataset	58
5.2	Dataset Split Statistics for CNN/DailyMail	58
5.3	Inference Parameters Configuration	60
5.4	Performance comparison on CNN/DailyMail test set	62
5.5	Ablation Study on Redundancy Criteria on CNN/DailyMail test set.	63
5.6	Performance comparison of model variants using different relevance and redundancy criteria combinations on CNN/DailyMail test set.	64
5.7	Component-wise comparison of relevance and redundancy on CNN/DailyMail	64
5.8	ROUGE scores of HIBERT with STAS criteria, and HIBERT with STAS plus Redundancy criteria on the CNN/DailyMail test set.	65
5.9	Qualitative comparison of a generated summary against the gold summary.	67

Listing of acronyms

ATS	Automatic Text Summarization
BERT	Bidirectional Encoder Representations from Transformers
BLEU	Bilingual Evaluation Understudy
BPE	Byte Pair Encoding
CNN	Convolutional Neural Network
GPT	Generative Pre-trained Transformer
GRU	Gated Recurrent Unit
HIBERT	Hierachical Bidirectional Encoder Representations from Transformers
LCS	Longest Common Subsequence
LLM	Large Language Model
LSA	Latent Semantic Analysis
LSTM	Long Short-Term Memory
MMR	Maximal Marginal Relevance
MSP	Masked Sentence Prediction
NLP	Natural Language Processing
PLM	Pre-trained Language Model
PMI	Pointwise Mutual Information
RNN	Recurrent Neural Network
ROUGE	Recall-Oriented Understudy for Gisting Evaluation
SQUAD	Stanford Question Answering Dataset
STAS	Sentence-level Transformer based Attentive Summarization
TF-IDF	Term Frequency-Inverse Document Frequency

Acknowledgments

The completion of this thesis has been possible thanks to the guidance and support of remarkable people around me.

I am sincerely grateful to Professor Tomaso Erseghe for his guidance, patience, and encouragement throughout this work. His insightful advice and dedicated supervision have been crucial in shaping this research and in helping me grow as a student.

I would also like to thank the Department of Mathematics at the University of Padova for offering the Master's program in Data Science. Its rigorous curriculum and stimulating environment have equipped me with the knowledge and skills essential for completing this thesis.

A very special thanks is reserved for the team at the IT Laboratories and Services of the Mathematics Department (Laboratori e Servizi Informatici-DM). Room Po36 became a second home to me, where I spent countless hours shaping this thesis and navigating my master's studies. Especially, my heartfelt gratitude goes to Andrea Simion, whose dedication in keeping the lab accessible and his kindness made a true difference in my academic journey.

Above all, I owe my deepest gratitude to my family. Their unconditional love, sacrifices, and unwavering faith in me have been the strongest source of strength in both my academic and personal life. This achievement is as much theirs as it is mine.

1

Introduction

1.1 PROBLEM STATEMENT AND MOTIVATION

The flow of modern life's information continues to accelerate, and the online content changes rapidly from day to day. Enormous volumes of textual data are generated daily, requiring people to read and process vast amounts of information. Therefore, the need for handling information efficiently is an important task. Consequently, nowadays, a central challenge in the field of natural language processing is how to leverage mathematical theory and computer technology to efficiently extract key information from massive text datasets and organize it into high-quality summaries.

Text summarization aims to produce a short and clear text that expresses the major ideas and important information of an original text. There are two main types of summarization methods: 1) abstractive summarization, which may generate new phrases that are not in the original text, and 2) extractive summarization, which extracts the most important sentences directly from the original text to create a summary. Both extractive and abstractive methods have their own advantages and disadvantages. Extractive methods are generally easier to implement and can produce summaries that are more faithful to the original text, but they may not capture the overall meaning of the text as well as abstractive methods. Abstractive methods can generate more concise and coherent summaries, but they are often more complex and have a higher computing cost. Therefore, the choice of summarization method depends on the specific re-

quirements of the task and the characteristics of the text being summarized. In this thesis, our goal is to highlight the most important sentences in a document, which is an extractive summarization task. We choose to focus on unsupervised methods, as this eliminates the need for labeled data during the training phase, making the approach more practical for real-world applications. Furthermore, this approach offers greater flexibility and adaptability to different domains and languages, as it does not rely on specific training data.

In the past decades, the field of automatic text summarization has undergone a clear progression: early statistical methods were gradually supplanted by supervised machine-learning and neural sequence-to-sequence models, and more recently by pretrained transformer models and large language models.[1]

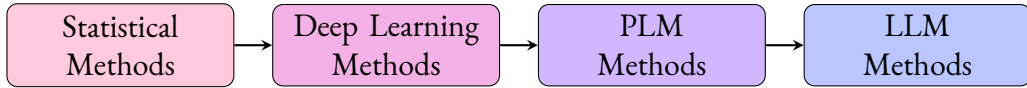


Figure 1.1: The Evolution of the Four Major Paradigms in Text Summarization [1]

The evolution of text summarization can be viewed as a sequence of solutions to the shortcomings of previous approaches (Figure 1.1). Early statistical methods were unsupervised and relied on heuristics such as TextRank [4] and feature engineering with TF-IDF. While simple and efficient, these models lacked the ability to capture semantic similarity or syntactic coherence beyond word co-occurrence.

Deep learning introduced sequence-to-sequence (seq2seq) architectures based on recurrent neural networks (RNNs) [5], which enabled the encoding of an input document into a continuous vector representation and its subsequent decoding into a summary. However, RNNs struggled with long-range dependencies due to the vanishing gradient problem. Long Short-Term Memory (LSTM) networks [6] addressed this by introducing gating mechanisms that allowed information to be retained across longer sequences. Nevertheless, LSTMs remained computationally expensive, required sequential processing, and still faced challenges when modeling very long documents or scaling to large datasets.

The Transformer architecture [7] marked a major breakthrough. Its self-attention mechanism allowed models to process entire sequences in parallel, capturing long-range dependencies and bidirectional context more efficiently than RNNs. Pre-trained Language Models (PLMs) such as BERT [8] further advanced the field by leveraging large-scale pretraining and transfer learning, enabling strong performance even with limited task-specific data.

Large Language Models (LLMs) such as GPT-3 [9] extended this paradigm with decoder-only architectures and billions of parameters, demonstrating emergent abilities like in-context learning, where tasks can be performed via few-shot prompting without parameter updates. For summarization, LLMs can generate high-quality summaries directly from prompts, but they introduce new challenges, including high computational requirements, hallucination risks, and variability depending on prompt design. Furthermore, their generalist nature often makes them less efficient or reliable for specialized domains, where smaller, fine-tuned PLMs may offer advantages in accuracy, efficiency, controllability, and deployment feasibility.

Considering this evolution, we focus on Hierarchical Bidirectional Transformers for extractive summarization. Hierarchical models are well-suited for long documents, as they capture both sentence-level and document-level representations, preserving global context while maintaining local coherence. Bidirectionality further enriches sentence representations by incorporating both past and future context. Finally, extractive summarization is chosen for its practical benefits: greater computational efficiency compared to abstractive methods, interpretability through direct traceability to the source, and reduced risk of hallucination, since the generated summary consists entirely of original text.

1.2 GOAL

The main goal of this thesis is to study the feasibility of using hierarchical models for extractive summarization. After studying the current state-of-the-art in extractive summarization, two models were the most promising candidates for this task: the HIBERT model[3] and the STAS model[10]. We will propose a framework to combine these two models and evaluate the performance of the proposed model on the CNN/Daily Mail dataset[11]. The proposed framework aims to leverage the strengths of both models to achieve better performance in extractive summarization tasks.

1.3 THESIS STRUCTURE OVERVIEW

Background: This chapter provides an overview of the theoretical background necessary to understand the concepts discussed in this thesis.

Related Work: This chapter reviews existing methods for text summarization that are related to this thesis.

Methodology: This chapter presents the proposed framework for extractive summarization, including the architecture of the model and the details of our methodology.

Results: This chapter presents the experimental setup, the evaluation metrics used to assess the performance of the proposed framework, the dataset used for evaluation, and the results obtained from the experiments. It also includes a comparison with existing methods, and an analysis of the results.

Conclusion: This chapter summarizes the main findings of the thesis, highlights the contributions made, and provides concluding remarks.

2

Theoretical Background

This chapter establishes the theoretical foundation for the thesis. We begin by examining the fundamentals of Automatic Text Summarization (ATS), including its definition, classification, and evaluation metrics. We then trace the evolution of summarization approaches through four major paradigms: statistical methods, deep learning methods, pre-trained language models, and large language models. Finally, we focus on extractive summarization and hierarchical architectures that form the core of this thesis.

2.1 FUNDAMENTALS OF TEXT SUMMARIZATION

2.1.1 DEFINITION AND MOTIVATION OF AUTOMATIC TEXT SUMMARIZATION (ATS)

Automatic Text Summarization (ATS) is the task of generating a concise and coherent summary of a longer text document while preserving its key information and overall meaning. ATS has gained significant attention in recent years due to the exponential growth of digital content and information overload.

ATS approaches are either extractive, abstractive, or hybrid. The extractive method selects the most important sentences directly from the original document to generate the summary. The abstractive method generates the summary with sentences that are different than the orig-

inal sentences. The hybrid approach combines both the extractive and abstractive methods.

The basic structure of ATS systems is shown in Figure 2.1. It typically comprises getting an input document or a set of documents, a pre-processing stage (e.g., cleaning the dataset and tokenization), a summarization module (extractive, abstractive, or hybrid), post-processing (e.g. reordering the selected sentences), and the final summary output.

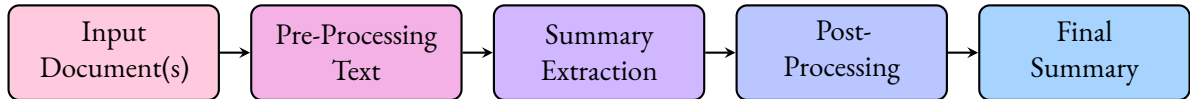


Figure 2.1: Model architecture of an automatic text summarizer.[1]

2.1.2 EXTRACTIVE VS. ABSTRACTIVE SUMMARIZATION PARADIGMS

ATS is broadly divided into two approaches:

- **Extractive Summarization:** This method works by identifying and selecting the most important sentences or phrases from the original text and concatenating them to form a summary. The output is composed entirely of segments from the source. The primary advantages of this approach are its factual consistency and grammatical correctness. It is generally faster and less prone to generating nonsensical or factually incorrect statements ("hallucinations").
- **Abstractive Summarization:** This method involves generating a new summary that paraphrases and synthesizes information from the source document, potentially using words and phrases not present in the original text. While this approach more closely mimics human summarization, it is significantly more complex and faces challenges in maintaining factual accuracy and coherence [12].

2.1.3 CLASSIFICATION OF ATS

ATS systems can be categorized according to multiple criteria, providing a comprehensive framework for understanding the diverse approaches in this field. The most widely recognized classification dimensions are illustrated in Figure 2.2[2].

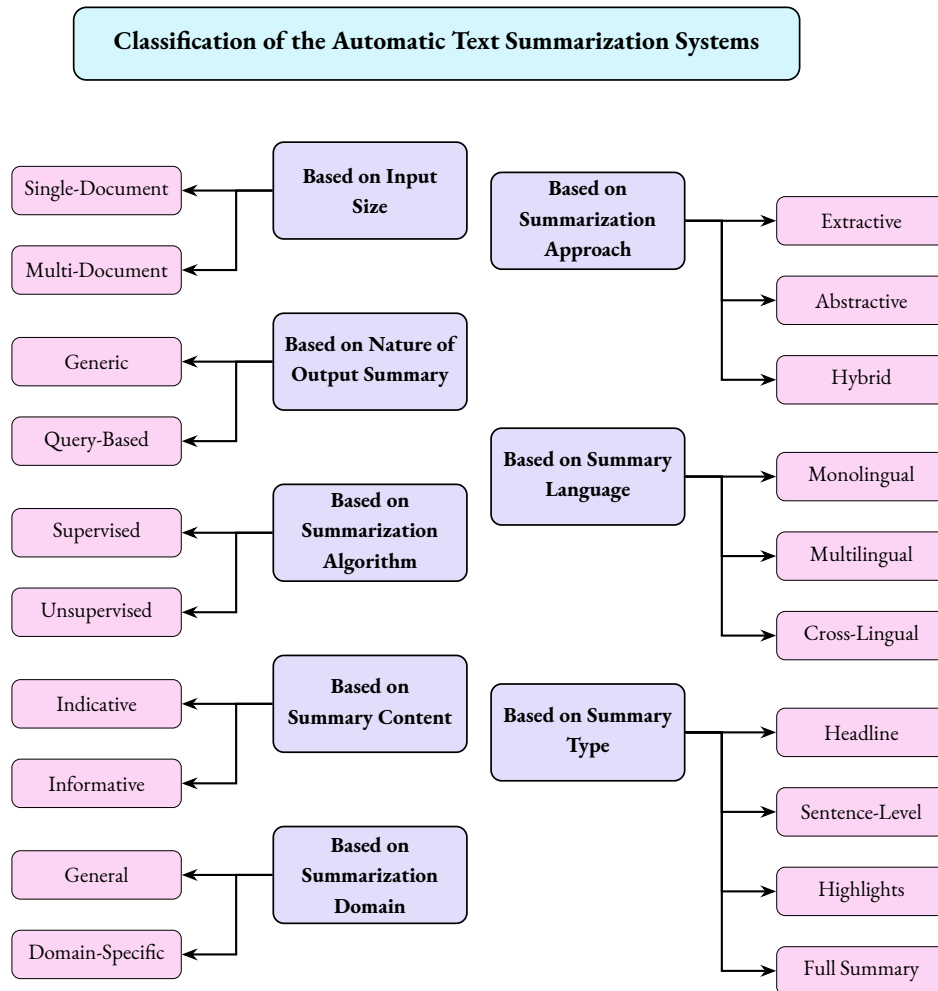


Figure 2.2: Classifications of the ATS systems based on the common characteristics.[2]

Based on the criteria mentioned in the figure, ATS systems can be classified along several different dimensions [2]:

- **Based on Input Size:** Summarization can be performed on a single document or multiple documents. Single-document summarization focuses on creating a summary from one text, while multi-document summarization aims to produce a single summary from a collection of related documents, which is more complex as it requires handling redundancy and synthesizing information from various sources [13].
- **Based on Nature of Output Summary:** This dimension distinguishes between generic and query-based summaries. A generic summary provides a general overview of the document's content, whereas a query-based summary is tailored to a specific user query, focusing only on the information relevant to that query [14].

- **Based on Summarization Algorithm:** This classification refers to the learning approach employed by the summarization system. Supervised methods require labeled training data to learn summarization patterns, while unsupervised methods derive summaries without explicit training on human-generated examples, often using statistical measures or graph-based algorithms.
- **Based on Summary Content:** This dimension categorizes summaries based on their intended purpose. Indicative summaries provide a brief overview to help users decide whether to read the full document, while informative summaries aim to cover all important information and serve as substitutes for the original text [15].
- **Based on Summarization Domain:** Summarization systems can be designed for general-purpose use across various domains or specialized for specific domains (such as biomedical, legal, or news texts). Domain-specific systems can leverage specialized knowledge and terminology to produce more accurate summaries within their target domain.
- **Based on Summarization Approach:** As discussed previously, this is one of the most fundamental distinctions. Extractive, Abstractive, and Hybrid approaches which are explained before.
- **Based on Summary Language:** This refers to the language(s) involved in the summarization process. Monolingual systems generate summaries in the same language as the source document, multilingual systems can handle documents in multiple languages, and cross-lingual systems generate summaries in a different language from the source document, combining summarization with machine translation [16].
- **Based on Summary Type:** This dimension categorizes the format and granularity of the generated summary. Options include headline summaries (very brief titles), sentence-level summaries (key sentences), highlights (important phrases or bullet points), and full summaries (comprehensive condensed versions of the original document).

We can also classify extractive and abstractive methods more based on their methodologies. Figure 2.3 illustrates the diversity of these approaches [2], which in the following sections, we will explore the theoretical foundations of some of these extractive methods. Later on, we will review the most important existing models for some of the extractive methods in the next chapter 3.

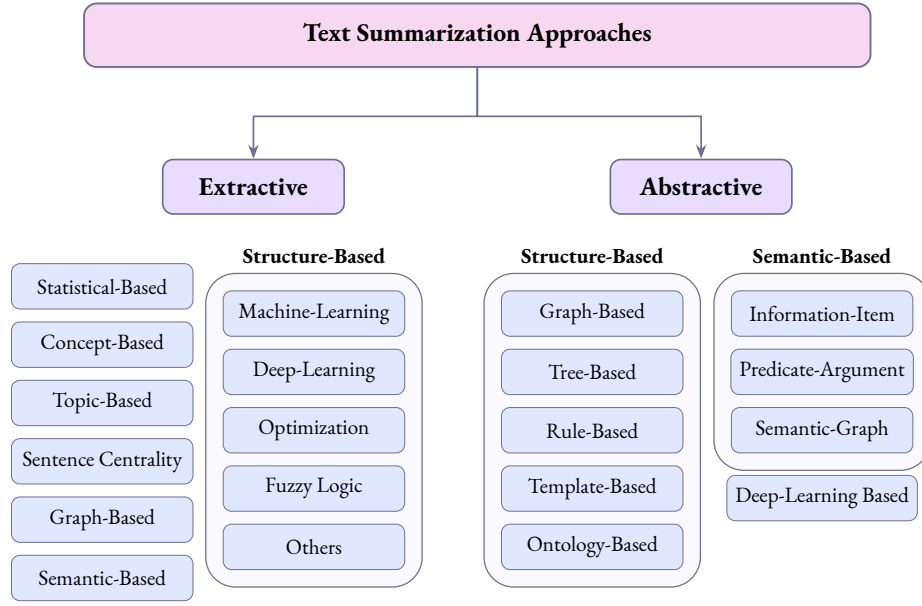


Figure 2.3: Automatic text summarization approaches and their associated methods.[2]

2.1.4 EVALUATION METRICS FOR SUMMARIZATION

Evaluating the quality of a machine-generated summary is a non-trivial task. An ideal summary should be fluent, coherent, concise, and, most importantly, factually consistent with the source document. Evaluation methods are broadly categorized into two types: intrinsic and extrinsic [14]. Intrinsic evaluation assesses the quality of the summary based on its content and linguistic properties, often by comparing it to a human-written reference summary. Extrinsic evaluation, on the other hand, measures the summary's usefulness in the context of a downstream task, such as question answering or information retrieval. This section focuses on the most common intrinsic evaluation metrics.

2.1.4.1 ROUGE (RECALL-ORIENTED UNDERSTUDY FOR GISTING EVALUATION)

ROUGE is the most widely used set of metrics for evaluating automatic summarization [17]. It measures the overlap between a machine-generated summary and one or more human-written reference summaries. The core idea is that a good summary should contain similar words and phrases to a human-created one.

2.1.4.1.1 ROUGE-N This metric measures the n-gram overlap. ROUGE-1 considers unigram overlap (individual words), while ROUGE-2 considers bigram overlap. ROUGE-N is

typically calculated based on recall, which measures how many of the n-grams in the reference summaries are captured by the system summary.

The formula for ROUGE-N recall is:

$$R_{\text{ROUGE-N}} = \frac{\sum_{S \in \{\text{RefSummaries}\}} \sum_{\text{gram}_n \in S} \text{Count}_{\text{match}}(\text{gram}_n)}{\sum_{S \in \{\text{RefSummaries}\}} \sum_{\text{gram}_n \in S} \text{Count}(\text{gram}_n)} \quad (2.1)$$

where $\text{Count}_{\text{match}}(\text{gram}_n)$ is the number of times an n-gram from the reference summary also appears in the system-generated summary, and $\text{Count}(\text{gram}_n)$ is the total number of n-grams in the reference summary. A corresponding precision score can also be calculated, and the F1-score is often reported:

$$F_{\text{I ROUGE-N}} = 2 \times \frac{P_{\text{ROUGE-N}} \times R_{\text{ROUGE-N}}}{P_{\text{ROUGE-N}} + R_{\text{ROUGE-N}}} \quad (2.2)$$

2.1.4.1.2 ROUGE-L This metric is based on the Longest Common Subsequence (LCS) between the system summary and the reference summary. It does not require consecutive matches but rather in-sequence matches, making it more sensitive to sentence-level structure. The recall and precision for ROUGE-L are calculated as follows:

$$R_{lcs} = \frac{\text{LCS}(X, Y)}{m} \quad (2.3)$$

$$P_{lcs} = \frac{\text{LCS}(X, Y)}{n} \quad (2.4)$$

where X is the reference summary of length m , Y is the system summary of length n , and $\text{LCS}(X, Y)$ is the length of their longest common subsequence. The F1-score is then computed:

$$F_{\text{I ROUGE-L}} = \frac{(1 + \beta^2) R_{lcs} P_{lcs}}{R_{lcs} + \beta^2 P_{lcs}} \quad (2.5)$$

Where $\beta = P_{LCS}/R_{LCS}$.

2.1.4.1.3 ROUGE-S This metric considers skip-bigrams, which are pairs of words in their original sentence order, but with any number of other words in between. This allows it to capture long-range dependencies and relationships between words more flexibly than ROUGE-N.

2.1.4.2 BLEU (BILINGUAL EVALUATION UNDERSTUDY)

BLEU [18] is a metric originally developed for evaluating machine translation quality, but it has also been applied to text summarization tasks. BLEU measures the quality of text by comparing n-gram overlap between the generated text and reference text(s), with an additional brevity penalty to penalize overly short outputs.

The BLEU score is calculated as:

$$\text{BLEU} = \text{BP} \times \exp \left(\sum_{n=1}^N w_n \log p_n \right) \quad (2.6)$$

where p_n is the precision of n-grams of length n , w_n is the weight for n-grams of length n (typically $w_n = 1/N$), and BP is the brevity penalty defined as:

$$\text{BP} = \begin{cases} 1 & \text{if } c > r \\ e^{(1-r/c)} & \text{if } c \leq r \end{cases} \quad (2.7)$$

where c is the length of the candidate translation and r is the effective reference corpus length.

While BLEU was not specifically designed for summarization evaluation, it provides a useful measure of n-gram overlap and has been used in some summarization studies, particularly for abstractive summarization where the generated text may differ significantly from the source document.

2.1.4.3 BERTScore

With the rise of contextual embeddings, newer metrics have been developed to overcome the limitations of n-gram-based overlap. BERTScore [19] is a prominent example. Instead of matching exact words, it computes the cosine similarity between the contextual embeddings (e.g., from BERT) of tokens in the candidate and reference summaries. This allows it to capture semantic similarity even when different words are used.

For each token x_i in the candidate summary, BERTScore finds the most similar token y_j in the reference summary, and vice versa. The recall, precision, and F1-score are then computed

as:

$$R_{\text{BERT}} = \frac{1}{|X|} \sum_{x_i \in X} \max_{y_j \in Y} \mathbf{x}_i^T \mathbf{y}_j \quad (2.8)$$

$$P_{\text{BERT}} = \frac{1}{|Y|} \sum_{y_j \in Y} \max_{x_i \in X} \mathbf{x}_i^T \mathbf{y}_j \quad (2.9)$$

$$F_{\text{BERT}} = 2 \frac{P_{\text{BERT}} \cdot R_{\text{BERT}}}{P_{\text{BERT}} + R_{\text{BERT}}} \quad (2.10)$$

where $\mathbf{x}_i$ and $\mathbf{y}_j$ are the contextual embeddings of the tokens.

2.1.4.4 HUMAN EVALUATION

While automatic metrics are essential for rapid development and comparison, human judgment remains the gold standard for evaluating summarization quality. Human evaluators are typically asked to rate summaries on several criteria, including fluency (whether the summary is grammatically correct and easy to read), coherence (whether the summary makes sense and has well-connected sentences), relevance and informativeness (whether the summary captures the most important information from the source document), and factual consistency (whether all information in the summary is factually correct with respect to the source). These evaluations are often conducted using Likert scales (e.g., 1-5) and provide a more holistic assessment of summary quality than automatic metrics alone [14].

2.1.4.5 COMPARATIVE ANALYSIS OF EVALUATION METRICS

Table 2.1 provides a comprehensive comparison of the most commonly used evaluation metrics for extractive text summarization, highlighting their respective strengths and limitations.

2.1.5 CHALLENGES IN SUMMARIZATION

There are some inherent challenges in summarization that impact both extractive and abstractive methods. First, content selection involves identifying the most relevant information from the source text, which is particularly challenging in long documents where important details may be buried within extensive content. Additionally, summaries often suffer from redundancy, where the same information is presented multiple times, a common issue especially in extractive methods that select sentences directly from the source without considering overlap.

Table 2.1: Comprehensive Comparison of Automatic Evaluation Metrics for Extractive Text Summarization[1]

Metric	Description	Primary Application	Key Advantages	Main Limitations
ROUGE (1, 2, L)	Evaluates n-gram overlap (unigrams, bigrams) and longest common sub-sequence between system and reference summaries	Standard benchmark for extractive summarization across all variants	Fast computation; captures lexical overlap and sequence structure; widely adopted; comprehensive coverage	Ignores semantic relationships; sensitive to vocabulary choices; limited paraphrase detection
BLEU	Precision-focused metric with brevity penalty, originally for machine translation	Cross-lingual summarization evaluation	Penalizes overly brief outputs; established in translation tasks	Precision bias may penalize comprehensive summaries; limited paraphrase detection
BERTScore	Leverages contextual embeddings for semantic similarity assessment	Modern neural summarization systems	Captures semantic equivalence; robust to vocabulary variations	Computationally expensive; requires pre-trained models; may overweight semantic similarity

Furthermore, generated summaries must be fluent and coherent, and while abstractive methods offer more flexibility in this regard, they can produce grammatically incorrect or nonsensical sentences that compromise readability. Finally, ensuring factual consistency so that the summary accurately reflects the source material is essential, particularly in domains like news summarization where factual accuracy is paramount and any distortion of information can have significant consequences.

2.2 STATISTICAL METHODS

The first paradigm in automatic text summarization was dominated by statistical and heuristic methods. These approaches, while limited compared to modern neural networks, established fundamental concepts and evaluation frameworks that continue to influence the field. This section covers the key statistical methods that formed the foundation of computational summarization from the 1950s through the early 2000s.

2.2.1 TERM FREQUENCY-INVERSE DOCUMENT FREQUENCY (TF-IDF)

One of the most foundational statistical measures in text processing is Term Frequency-Inverse Document Frequency (TF-IDF) [20]. Originally developed for information retrieval, TF-IDF became a base of extractive summarization systems by providing a principled way to identify the most important terms in a document.

The TF-IDF score for a term t in document d from a collection D is calculated as:

$$\text{TF-IDF}(t, d, D) = \text{TF}(t, d) \times \text{IDF}(t, D) \quad (2.11)$$

where the term frequency $\text{TF}(t, d)$ measures how often term t appears in document d :

$$\text{TF}(t, d) = \frac{\text{count}(t, d)}{\sum_{t' \in d} \text{count}(t', d)} \quad (2.12)$$

and the inverse document frequency $\text{IDF}(t, D)$ measures the rarity of the term across the entire collection:

$$\text{IDF}(t, D) = \log \left(\frac{|D|}{|\{d \in D : t \in d\}|} \right) \quad (2.13)$$

In the context of summarization, sentences containing terms with high TF-IDF scores are considered more informative and thus more likely to be included in the summary. This approach assumes that important sentences contain terms that are frequent within the document but rare across the broader collection, making them distinctive and informative [21].

2.2.2 MAXIMAL MARGINAL RELEVANCE (MMR)

Maximal Marginal Relevance [22] addresses a critical limitation of purely relevance-based selection: redundancy. MMR provides a framework for selecting sentences that are both relevant to the main topic and diverse from already selected sentences.

The MMR criterion is defined as:

$$\text{MMR} = \arg \max_{s_i \in R \setminus S} \left[\lambda \cdot \text{Sim}_1(s_i, Q) - (1 - \lambda) \cdot \max_{s_j \in S} \text{Sim}_2(s_i, s_j) \right] \quad (2.14)$$

where: R is the set of all candidate sentences, S is the set of already selected sentences, Q represents the query or document representation, λ controls the trade-off between relevance and diversity, Sim_1 measures relevance similarity, and Sim_2 measures redundancy similarity.

This iterative selection process ensures that each newly selected sentence provides maximal additional information while maintaining relevance to the document’s main themes.

2.2.3 GRAPH-BASED METHODS

Graph-based approaches model documents as networks where nodes represent textual units (words, sentences, or paragraphs) and edges represent relationships between them. These methods leverage graph theory algorithms to identify the most central or important elements.

2.2.3.1 TEXTRANK

TextRank [4] adapts Google’s PageRank algorithm for text summarization. In this approach, sentences are treated as nodes in a graph, and edges represent semantic similarity between sentences.

The similarity between two sentences s_i and s_j is typically calculated using cosine similarity:

$$\text{sim}(s_i, s_j) = \frac{\sum_{w \in s_i \cap s_j} \log(\text{freq}(w)) \times \log(\text{freq}(w))}{\log(|s_i|) \times \log(|s_j|)} \quad (2.15)$$

The TextRank score for each sentence is computed iteratively:

$$\text{TR}(s_i) = (1 - d) + d \times \sum_{s_j \in \text{adj}(s_i)} \frac{\text{sim}(s_i, s_j)}{\sum_{s_k \in \text{adj}(s_j)} \text{sim}(s_j, s_k)} \times \text{TR}(s_j) \quad (2.16)$$

where d is a damping factor (typically 0.85) and $\text{adj}(s_i)$ represents sentences adjacent to s_i in the graph.

2.2.3.2 LEXRANK

LexRank [23] is another graph-based method that uses eigenvector centrality to identify important sentences. It constructs a similarity matrix where each entry represents the cosine similarity between sentence pairs, then applies the power method to find the principal eigenvector, which indicates sentence importance.

The algorithm begins by constructing a similarity matrix $\mathbf{M}$ where each entry M_{ij} represents the cosine similarity between sentences s_i and s_j :

$$M_{ij} = \text{cosine}(s_i, s_j) = \frac{\mathbf{v}_i \cdot \mathbf{v}_j}{|\mathbf{v}_i| |\mathbf{v}_j|} \quad (2.17)$$

where $\mathbf{v}_i$ and $\mathbf{v}_j$ are vector representations of sentences s_i and s_j (typically TF-IDF weighted). A binary adjacency matrix $\mathbf{A}$ is then created by applying a threshold t to the similarity matrix:

$$A_{ij} = \begin{cases} 1 & \text{if } M_{ij} > t \text{ and } i \neq j \\ 0 & \text{otherwise} \end{cases} \quad (2.18)$$

The degree-normalized transition matrix $\mathbf{P}$ is computed as:

$$P_{ij} = \frac{A_{ij}}{\sum_{k=1}^n A_{ik}} \quad (2.19)$$

The LexRank score for each sentence is computed as the stationary distribution of this Markov chain. The LexRank vector $\mathbf{x}$ satisfies:

$$\mathbf{x} = \mathbf{P}^T \mathbf{x} \quad (2.20)$$

This can be solved using the power method, where the LexRank scores converge to the principal eigenvector of $\mathbf{P}^T$. The algorithm iteratively updates:

$$\mathbf{x}^{(k+1)} = \mathbf{P}^T \mathbf{x}^{(k)} \quad (2.21)$$

until convergence. Sentences with higher LexRank scores are considered more central to the document's content and are prioritized for inclusion in the summary.

Similar to TextRank, LexRank can also incorporate a damping factor d to handle disconnected components in the graph:

$$\text{LexRank}(s_i) = \frac{1-d}{n} + d \sum_{j=1}^n P_{ji} \cdot \text{LexRank}(s_j) \quad (2.22)$$

where n is the total number of sentences and d is typically set to 0.85.

2.2.4 LIMITATIONS OF CLASSICAL STATISTICAL METHODS

While these classical approaches laid important groundwork for automatic summarization, they suffer from several fundamental limitations. First, statistical methods operate primarily at the surface level, focusing on word frequencies and positional features without deep semantic understanding of the text content. Additionally, most methods consider only local context within sentences, thereby missing broader discourse-level relationships that are crucial for coherent

summarization. Furthermore, their success depends heavily on manually crafted features that may not generalize well across different domains or text types. These approaches are also predominantly extractive in nature, which inherently limits their ability to generate coherent, abstractive summaries that require paraphrasing and synthesis of information. Finally, many of the features employed, such as cue words and positional indicators, are language-specific and don't transfer well across different languages, limiting their applicability in multilingual contexts.

2.3 DEEP LEARNING METHODS

The second paradigm emerged with the application of deep neural networks to text summarization. This era was characterized by the introduction of sequence-to-sequence models, recurrent neural networks, and used architectures that began to address the semantic limitations of statistical approaches.

2.3.1 RECURRENT NEURAL NETWORKS FOR SUMMARIZATION

The transition from statistical to neural methods began with the application of Recurrent Neural Networks (RNNs) to text summarization. Unlike statistical methods that relied on hand-crafted features, RNNs could learn representations directly from data.

RNNs are designed to process sequential data by maintaining an internal state that captures information from previous elements. The core computation at each time step t is:

$$h_t = \phi(x_t W_{xh} + h_{t-1} W_{hh} + b_h) \quad (2.23)$$

$$o_t = h_t W_{ho} + b_o \quad (2.24)$$

where h_t is the hidden state at time step t , x_t is the input vector, o_t is the output, and ϕ is typically the tanh activation function. The weight matrices are: W_{xh} (input-to-hidden weights), W_{hh} (hidden-to-hidden recurrent weights), W_{ho} (hidden-to-output weights), and b_h, b_o are the bias vectors for the hidden and output layers respectively.

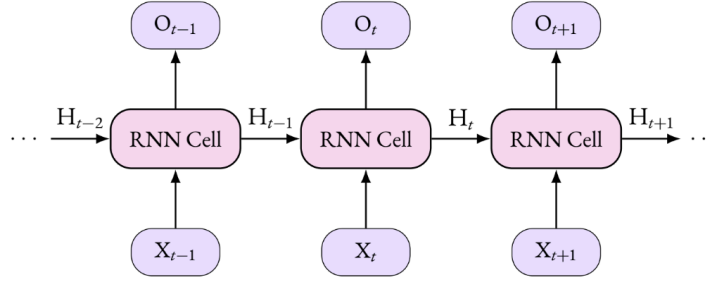


Figure 2.4: The architecture of a Recurrent Neural Network, unrolled through time. At each step t , the cell processes the input X_t and the previous hidden state H_{t-1} to produce the output O_t and the new hidden state H_t

However, standard RNNs suffer from the vanishing gradient problem [24], where information from earlier time steps progressively weakens as it propagates through the network, making it difficult to capture long-range dependencies in sequences. This fundamental limitation led to the development of gated architectures like Long Short-Term Memory (LSTM) networks [6] and Gated Recurrent Units (GRU) [25]. LSTMs address this issue through sophisticated gating mechanisms that selectively control what information to retain, update, or discard, allowing the network to maintain relevant context over extended sequences. GRUs provide a simplified alternative that combines the forget and input gates of LSTMs into a single update gate, achieving similar performance with reduced computational complexity while still effectively handling long-range dependencies.

2.3.2 PERSISTENT CONTEXT LIMITATIONS

Despite their improvements over standard RNNs, LSTM and GRU architectures still face fundamental limitations when applied to text summarization. First, while gating mechanisms help preserve information, the hidden state still has finite capacity, meaning that for very long documents, early information may still be forgotten or compressed, limiting the model’s ability to maintain coherent global context throughout the entire document. Additionally, information from the beginning of a long document must be compressed and passed through all intermediate time steps to influence decisions at the end, creating a representational bottleneck where nuanced relationships between distant sentences may be lost. Furthermore, while gating mechanisms alleviate vanishing gradients, they don’t eliminate the problem entirely, as training signals from distant parts of the sequence may still be weakened, making it difficult to learn complex long-range dependencies crucial for coherent summarization. Finally, the inherent sequential nature means that processing long documents remains computationally expensive

and cannot be fully parallelized, limiting scalability for real-world applications with lengthy texts. These context-related limitations motivated the development of attention mechanisms and, eventually, the Transformer architecture that could model long-range dependencies more effectively while enabling parallel computation.

2.4 PRE-TRAINED LANGUAGE MODELS

The third paradigm was ushered in by the Transformer architecture and the pre-training paradigm. This era revolutionized NLP by introducing models that could be pre-trained on massive unlabeled corpora and then fine-tuned for specific tasks, achieving unprecedented performance across various NLP tasks including text summarization.

2.4.1 THE TRANSFORMER ARCHITECTURE

In 2017, Vaswani et al. introduced the Transformer, a deep learning architecture that completely revolutionized the field of NLP [7]. It dispensed with recurrence entirely, proposing a model based solely on attention mechanisms. This shift addressed the limitations of RNNs, enabling massive parallelization and setting a new standard for modeling long-range dependencies.

2.4.2 THE SELF-ATTENTION MECHANISM

The core innovation of the Transformer is the self-attention (or intra-attention) mechanism. Instead of processing a sentence word-by-word, self-attention allows the model to weigh the importance of all words in the input sequence simultaneously and draw context from any of them directly.

To achieve this, each input vector is projected into three distinct representations: a Query (Q), a Key (K), and a Value (V). The intuition is that the Query represents the current word's request for information, the Keys from all other words advertise what information they hold, and the Values contain the actual content to be passed on.

The attention score is calculated by taking the dot product of the Query vector with the Key vectors of all other words. This score is then scaled, passed through a softmax function to create a probability distribution, and used to compute a weighted sum of the Value vectors. The most widely used form is Scaled Dot-Product Attention:

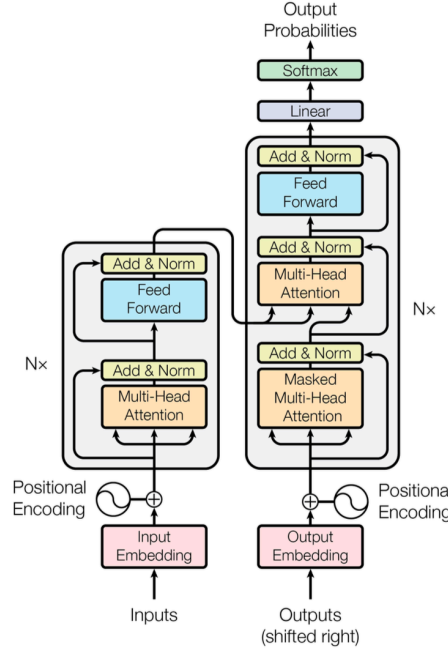


Figure 2.5: The Transformer architecture proposed by Vaswani et al.[7]

$$\text{Attention}(Q, K, V) = \text{softmax} \left(\frac{QK^T}{\sqrt{d_k}} \right) V \quad (2.25)$$

Here, Q , K , and V are matrices containing the queries, keys, and values for all tokens in the sequence. The scaling factor $\sqrt{d_k}$, where d_k is the dimension of the keys, is crucial for stabilizing gradients during training [7]. This mechanism allows the model to look at an entire sequence at once, creating rich, context-aware representations for each token.

2.4.3 MULTI-HEAD ATTENTION

To further enhance the model's capabilities, the Transformer employs Multi-Head Attention. Instead of performing a single attention function, this mechanism runs multiple scaled dot-product attention operations—the "heads"—in parallel. Each head learns to attend to different aspects of the sequence by using different, learned linear projections for its queries, keys, and values.

$$\text{head}_i = \text{Attention}(QW_i^Q, KW_i^K, VW_i^V) \quad (2.26)$$

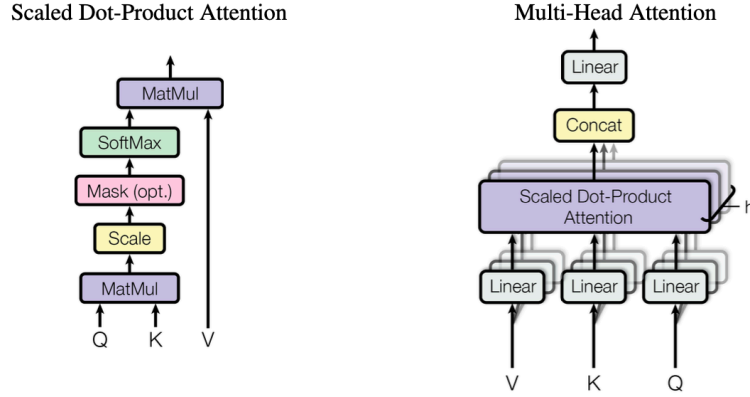


Figure 2.6: (left) Scaled Dot-Product Attention. (right) Multi-Head Attention[7]

The outputs of these parallel heads are then concatenated and linearly projected back to the original dimension.

$$\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, \dots, \text{head}_h)W^O \quad (2.27)$$

This allows the model to jointly attend to information from different representation subspaces. For example, one head might focus on syntactic relationships while another captures semantic similarities.2.6

2.4.4 ENCODER-DECODER STRUCTURE

The original Transformer model consists of an encoder and a decoder, each a stack of identical layers.

The Encoder stack processes the entire input sequence to generate a rich contextual representation. Each encoder layer contains two sub-layers: a multi-head self-attention mechanism and a position-wise fully connected feed-forward network.

The Decoder stack generates the output sequence one token at a time (autoregressively). In addition to the two sub-layers found in the encoder, the decoder inserts a third sub-layer that performs multi-head attention over the encoder’s output, allowing it to focus on relevant parts of the input sequence.

A residual connection [26] followed by layer normalization [27] is applied around each of the sub-layers, which helps in training deep models:

$$\text{LayerNorm}(x + \text{Sublayer}(x)) \quad (2.28)$$

The feed-forward network (FFN) within each layer provides additional non-linear transformations:

$$\text{FFN}(x) = \text{Activation}(xW_1 + b_1)W_2 + b_2 \quad (2.29)$$

While the original Transformer used ReLU activation ($\max(0, x)$), modern implementations often employ GELU (Gaussian Error Linear Unit) activation, which provides smoother gradients and has been shown to improve performance in many NLP tasks. GELU is defined as:

$$\text{GELU}(x) = x \cdot \Phi(x) = x \cdot \frac{1}{2}[1 + \text{erf}(x/\sqrt{2})] \quad (2.30)$$

where $\Phi(x)$ is the cumulative distribution function of the standard normal distribution.

2.4.5 POSITIONAL ENCODING

Since the Transformer contains no recurrence or convolution, it has no inherent sense of word order. To remedy this, positional encodings are added to the input embeddings at the bottom of the encoder and decoder stacks. These are vectors that provide information about the position of each token in the sequence. Vaswani et al. (2017) proposed using sine and cosine functions of different frequencies:

$$PE_{(pos, 2i)} = \sin(pos/10000^{2i/d_{\text{model}}}) \quad (2.31)$$

$$PE_{(pos, 2i+1)} = \cos(pos/10000^{2i/d_{\text{model}}}) \quad (2.32)$$

Here, pos is the token's position, i is the dimension within the embedding vector, and d_{model} is the model's embedding dimension. This technique allows the model to learn to attend to relative and absolute positions.

2.4.6 INPUT REPRESENTATION AND EMBEDDINGS

The Transformer processes discrete tokens by converting them into continuous vector representations. The input to a Transformer consists of two types of embeddings that are summed together. First, **Token Embeddings**: Each token (word or subword) is mapped to a learnable

dense vector representation of dimension d_{model} . Modern transformers typically use subword tokenization schemes such as Byte-Pair Encoding (BPE) [28] or WordPiece to handle out-of-vocabulary words and improve representation efficiency. Second, **Positional Embeddings**: As described above, these provide information about the position of tokens in the sequence.

The final input representation is computed as:

$$\text{Input}(x) = \text{TokenEmbedding}(x) + \text{PositionalEmbedding}(x) \quad (2.33)$$

2.4.7 TRAINING OBJECTIVES AND PRE-TRAINING

Pre-trained language models like BERT[8] use self-supervised learning objectives that enable them to learn rich linguistic representations from large unlabeled datasets:

- **Masked Language Modeling (MLM)**: During pre-training, a percentage of input tokens are randomly masked, and the model learns to predict the original tokens based on the surrounding context. This bidirectional training objective allows the model to learn representations that capture both left and right context.
- **Next Sentence Prediction (NSP)**: The model learns to predict whether two sentences follow each other in the original text, helping it understand sentence-level relationships. While less critical than MLM, NSP provides additional training signal for document-level understanding.

This pre-training approach enables transfer learning, where the model first learns general language understanding from massive datasets, then can be fine-tuned on specific downstream tasks with much smaller labeled datasets.

2.4.8 ADVANTAGES OVER RNNs

Transformers offer three primary advantages over recurrent models: parallelization (since computations for each token are not dependent on the previous token's output, the entire sequence can be processed in parallel, leading to dramatically faster training times), superior handling of long-range dependencies (the path length between any two positions in the sequence is constant $O(1)$ via the self-attention mechanism, whereas in RNNs this path length is linear $O(n)$, making gradient propagation over long distances much harder), and more efficient context learning (the attention mechanism allows direct modeling of relationships between any two words in a sentence, resulting in more powerful and contextually aware representations).

2.5 LARGE LANGUAGE MODELS

The fourth and current paradigm is represented by Large Language Models (LLMs) that have hundreds of billions to trillions of parameters. These models are trained on massive text corpora using autoregressive language modeling objectives and demonstrate remarkable emergent capabilities that were not explicitly programmed for. LLMs can perform text summarization through prompting techniques, where users provide instructions and examples directly in the input, eliminating the need for task-specific fine-tuning and enabling rapid adaptation to various summarization styles and domains.

However, for the specific focus of this thesis on hierarchical extractive summarization, we prioritize specialized pre-trained models over LLMs for several critical reasons. First, the computational requirements of LLMs can be impractical, requiring better GPU resources. Second, LLMs are primarily designed for abstractive generation and lack the explicit structural modeling of sentence-document hierarchies that is essential for extractive summarization tasks. Finally, specialized hierarchical architectures provide more controlled, predictable, and interpretable approaches to sentence selection, with superior performance on established benchmarks while maintaining computational efficiency.

2.6 HIERARCHICAL ARCHITECTURES FOR EXTRACTIVE SUMMARIZATION

First, it is important to state that, by hierarchical architecture in this thesis, we specifically mean that the hierarchical transformer has a sentence-level encoder and a document-level encoder[10, 3]. The transformers are based on Vaswani et al[7] and have multiple layers, with each layer having a multi-head self-attentive sub-layer that is followed by a feed-forward sub-layer with residual connections and layer normalizations[3].

The hierarchical architecture is designed to capture the multi-level structure of text, where sentences are processed individually before being integrated into a document-level representation. This allows for more effective modeling of the relationships between sentences and their context within the document, ultimately leading to improved performance on extractive summarization tasks.

The general pipeline for the hierarchical architecture involves several key steps2.7. Sentences are first tokenized and embedded, then encoded to produce sentence-level vectors. These vec-

tors are augmented with sentence position embeddings and are fed to a document-level Transformer to get context-aware sentence representations that model inter-sentence dependencies. Furthermore, based on supervised or unsupervised summarization, the context-aware representations are either used to assign a True or False label for each sentence, where True means this sentence should be included in the summary[3]. Or based on the pretraining objectives and the context-aware representations, they can be used to rank sentences for extractive unsupervised selection[10].

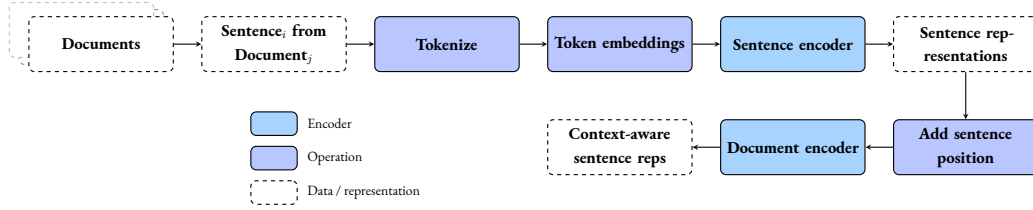


Figure 2.7: Hierarchical architecture pipeline for extractive summarization. Each sentence is tokenized and embedded (word + positional), encoded by a sentence-level Transformer to yield a sentence vector. The sequence representations are then processed by a document-level Transformer to produce context-aware representations.

2.6.1 PROBLEM FORMULATION

Consider a document $D = \{S_1, S_2, \dots, S_{|D|}\}$, where $S_i = \{w_1, w_2, \dots, w_{|S_i|}\}$ is a sentence in the document and w_j is a word in the sentence[3]. The goal of extractive summarization is to select a subset of sentences from the document that best represent its content.

Then, the embedding E_i of each sentence S_i is obtained by summing the token embeddings and positional embeddings of the words in the sentence. The sentence-encoder applies on the embedding E_i to produce a sequence of hidden representations:

$$\text{Sent-Encoder}(E_i) = (h_1^{(i)}, h_2^{(i)}, \dots, h_{|S_i|}^{(i)}) \quad (2.34)$$

Then sentence positions are added to the sentence representations to form the input to the document-level encoder:

$$\text{Doc-Encoder}(H_i) = (d_1^{(i)}, d_2^{(i)}, \dots, d_{|D|}^{(i)}) \quad (2.35)$$

This leads to the final context-aware sentence representations $(d_1^{(i)}, d_2^{(i)}, \dots, d_{|D|}^{(i)})$.

In the next subsection, we will discuss how these context-aware representations are used in two different hierarchical models for extractive summarization.

2.6.2 BASE MODELS

The two hierarchical models considered in this thesis are HIBERT[3] and STAS[10].

HIBERT is a supervised hierarchical extractive summarization model that does a binary classification to predict whether each sentence should be included in the summary. HIBERT’s pre-training involves masked sentence prediction, since it wants to learn the representation of a document, where its basic units are sentences.

The probability that sentence S_i is included in the summary is given by:

$$p(y_i \mid \mathcal{D}) = \text{softmax}(\mathbf{W}^S \vec{\mathbf{d}}_i)$$

where, $\vec{\mathbf{d}}_i \in \mathbb{R}^d$ is the context-aware representation of sentence S_i from the encoder, and $\mathbf{W}^S \in \mathbb{R}^{2 \times d}$ is a learnable projection matrix for the binary classification task.

STAS is an unsupervised hierarchical extractive summarization model that uses a ranking mechanism to score sentences based on their importance and selects the top-ranked sentences for the summary. STAS’s pre-training involves masked sentences prediction and sentence shuffling, since they want the document encoder to learn to select sentences based on their contents rather than positions. STAS employs context-aware sentence representations from a pre-trained hierarchical transformer to calculate a sentence’s importance score. This score combines 1) its likelihood of being reconstructed via the Masked Sentence Prediction objective and 2) the aggregated attention it receives from other sentences, derived from the model’s self-attention patterns.

3

Related Works

Text summarization has evolved through several distinct paradigms, each addressing limitations of previous approaches while introducing new capabilities. The field has progressed from early statistical methods through deep learning approaches to the current era of pre-trained language models.

This chapter reviews the literature most relevant to our work on unsupervised extractive text summarization using hierarchical transformer architectures. We organize our review around three main themes: the evolution of text summarization paradigms with particular emphasis on unsupervised approaches, the development of hierarchical transformer models for document understanding, and the specific models that form the foundation of our proposed method: HIBERT[3] and STAS[10].

3.1 EVOLUTION OF TEXT SUMMARIZATION PARADIGMS

3.1.1 STATISTICAL AND GRAPH-BASED APPROACHES

Early unsupervised extractive methods were predominantly statistical, relying on heuristics to identify salient sentences. They were usually built on the principle that important content is often signaled by frequently used words (lexical prominence) and its location within the document. For instance, Luhn’s method scored sentences based on the frequency and clustering of significant words, assuming that high-frequency terms signal thematic importance [21]. Ed-

mundson expanded this by incorporating multiple features: cue words (e.g., "in conclusion"), title and heading overlap, and sentence position, with scores calculated as a weighted sum of these indicators [29]. These pioneering approaches paved the way for more standardized techniques like TF-IDF, which assigns weights to words based on their term frequency and inverse document frequency, allowing sentence scores to be derived from the words they contain [20].

Subsequent methods aimed to capture document-level semantics more holistically. Matrix factorization techniques, particularly Latent Semantic Analysis (LSA), projected documents into a lower-dimensional topic space using Singular Value Decomposition (SVD) [30, 31]. By representing sentences in this latent space, LSA could identify salient content by selecting sentences that best represented the document's primary topics, moving beyond simple word frequencies to capture co-occurrence patterns [32]. Another key approach, centroid-based summarization, first computes a document centroid—a vector representing the weighted average of all words in the document. Sentences are then scored based on their similarity to this centroid, with the highest-scoring sentences selected for the summary [33].

Graph-based ranking algorithms offered a relational perspective, modeling the document as a network of interconnected sentences. In this paradigm, a sentence's importance is determined by its connections to other important sentences. TextRank [4] famously adapted the PageRank algorithm to sentence graphs, where sentences are nodes and edges are weighted by lexical similarity. Similarly, LexRank [23] uses eigenvector centrality on a cosine similarity graph to identify the most central sentences. To mitigate the redundancy inherent in centrality-based methods, Maximal Marginal Relevance (MMR) was introduced as a selection strategy [22]. MMR iteratively selects sentences that are both relevant to the document and dissimilar to sentences already chosen, explicitly balancing informativeness and novelty. While powerful and efficient, these statistical and graph-based methods were fundamentally limited by their reliance on surface-level features (e.g., word overlap), making them susceptible to issues with paraphrase and limiting their semantic understanding; a gap that deep learning models would later address.

3.1.2 DEEP LEARNING ERA

The introduction of deep learning methods brought significant improvements in semantic understanding and long-range dependency modeling, shifting the focus from handcrafted features to learned representations. This era is characterized by the application of neural networks, particularly Recurrent Neural Networks (RNNs), to model the sequential nature of text.

Early deep learning approaches bifurcated into abstractive and extractive methods. Abstractive summarization was revitalized by the application of sequence-to-sequence (seq2seq) models, first introduced for machine translation. Rush et al. [34] pioneered this by using an attention-based encoder-decoder framework to generate abstractive summaries. Subsequent work by Nallapati et al. [35] extended this model with features like pointer-generator networks to address challenges of factual inaccuracy and out-of-vocabulary words. While powerful, these abstractive models were computationally expensive, difficult to train, and prone to generating non-factual statements.

In parallel, neural methods for extractive summarization emerged, replacing statistical heuristics with learned sentence representations. One of the earliest influential models was proposed by Cheng and Lapata [36], who used a hierarchical RNN to first encode words and then sentences, creating a document representation that informed sentence selection. This approach framed extractive summarization as a sentence ranking task learned from data.

Building on this, SummaRuNNer [37] formulated extractive summarization as a sequence labeling problem. It employed a bidirectional RNN to process the document, classifying each sentence as either part of the summary or not. The model’s decisions were based on a combination of sentence content, document context, and abstract features like position and novelty. This sequence-to-sequence labeling approach became a standard for supervised extractive summarization. A notable limitation of these early RNN-based models, however, was their struggle with long-range dependencies and the vanishing gradient problem, which made it difficult to effectively process long documents; a key challenge that the subsequent Transformer-based models would address.

3.1.3 THE PRE-TRAINED LANGUAGE MODEL ERA

The introduction of the Transformer architecture in “Attention is All You Need” by Vaswani et al. (2017) marked a historical moment in natural language processing [7]. By replacing recurrent layers with a self-attention mechanism, the Transformer could model long-range dependencies more effectively and process sequences in parallel, overcoming the limitations of RNNs. This architectural innovation paved the way for pre-trained language models (PLMs), which are trained on vast text corpora and can be fine-tuned for a wide range of downstream tasks, including summarization. These models learn rich, contextualized representations of language that capture complex semantic and syntactic relationships.

3.1.3.1 BERT AND ITS IMPACT

Among the first and most influential PLMs was BERT (Bidirectional Encoder Representations from Transformers) [8]. BERT introduced a novel pre-training objective, Masked Language Modeling (MLM), which allowed it to learn deep bidirectional representations by predicting randomly masked tokens in a sequence. This approach enabled BERT to gain a more profound contextual understanding of language compared to previous unidirectional models. Its powerful representations could be readily adapted to various tasks, and it quickly became a foundational component for subsequent summarization models.

3.1.3.2 ADAPTING PLMS FOR SUMMARIZATION

The success of BERT in understanding tasks naturally led researchers to adapt it for generation. However, its architecture, designed for encoding rather than generation, posed a significant challenge. A pioneering effort to overcome this was BERTSUM [38], which modified the BERT architecture specifically for document-level understanding and extractive summarization. It introduced special tokens ([CLS]) to mark the beginning of each sentence, used interval segment embeddings to distinguish between sentences, and added inter-sentence Transformer layers to explicitly model relationships between them. BERTSUM achieved state-of-the-art results and demonstrated the effectiveness of architecturally adapting pre-trained models for summarization tasks. Following BERTSUM’s success, researchers explored different directions for improving summarization with pre-trained models.

3.2 UNSUPERVISED EXTRACTIVE SUMMARIZATION WITH PRE-TRAINED MODELS

The integration of powerful pre-trained language models with unsupervised methods has emerged as a promising direction, addressing the persistent challenge of limited labeled data while leveraging deep contextual understanding. These approaches enhance traditional unsupervised techniques by replacing shallow text representations with rich, contextualized embeddings from models like BERT.

3.2.1 GRAPH-BASED METHODS ENHANCED WITH PRE-TRAINED EMBEDDINGS

Modern graph-based approaches have significantly improved by replacing traditional TF-IDF representations with rich pre-trained embeddings. This allows for a more nuanced understanding of semantic similarity between sentences, leading to more accurate and coherent summaries.

3.2.1.1 PACSUM

PACSUM (Position-Augmented Centrality based Summarization) [39] enhances unsupervised extractive summarization by integrating contextual embeddings with positional awareness. The method addresses the limitations of traditional graph-based approaches, namely shallow sentence representations and the neglect of positional information. PACSUM achieves this by using BERT to generate rich, contextualized sentence embeddings, where each sentence is represented by its [CLS] token. It then constructs a position-aware graph that incorporates both semantic similarity and positional bias, recognizing that sentences appearing earlier in a document are often more salient. A discourse-aware weighting scheme gives higher importance to sentences in key positions, and a modified PageRank algorithm, influenced by both semantic connectivity and positional importance, is applied to rank the sentences.

3.2.1.2 GUSUM

GUSUM (Graph-based Unsupervised Summarization) [40] also advances graph-based methods by using Sentence-BERT to generate contextually rich sentence embeddings, which capture semantic nuances more effectively than traditional representations. The method constructs a document graph where edge weights are based on the cosine similarity between these embeddings, resulting in a more semantically meaningful structure. GUSUM applies a modified PageRank algorithm that incorporates sentence position and length normalization to better identify salient sentences. Finally, it fuses multiple ranking signals, including graph centrality, sentence position, and content overlap with the document title, to produce the final summary.

3.2.2 INFORMATION-THEORETIC APPROACHES

3.2.2.1 PMI-BASED RANKING

The Pointwise Mutual Information (PMI) approach [41] offers a formal framework for unsupervised extractive summarization grounded in information theory. This method leverages a pre-trained language model, such as GPT-2, to estimate sentence relevance and redundancy without labeled data. The core idea is to select sentences that are both informative and novel.

Relevance is measured by how much a given sentence increases the probability of the entire document, effectively identifying sentences that are central to the document’s meaning. Redundancy is quantified by measuring the semantic overlap between sentence pairs, ensuring that a new sentence is not selected if its content can be inferred from already chosen sentences. The selection process greedily maximizes relevance while penalizing redundancy, providing a principled way to balance informativeness and conciseness in the final summary.

3.2.3 MULTI-MODAL AND ENSEMBLE APPROACHES

3.2.3.1 RANKSUM

RANKSUM [42] demonstrates the effectiveness of combining multiple unsupervised signals through rank fusion. Instead of relying on a single heuristic, it integrates topic-based features from Latent Dirichlet Allocation (LDA), semantic features from sentence embeddings, keyword-based features from graph centrality, and positional features. Although the fusion weights are learned from a small labeled dataset, the model generalizes well across different domains, making it practically applicable in many unsupervised settings. This ensemble approach highlights the power of integrating diverse information sources to create more robust and accurate summaries.

3.3 HIERARCHICAL TRANSFORMER ARCHITECTURES

Human readers naturally leverage hierarchical cues (section headers, paragraph boundaries) to understand information flow and identify key themes. Hierarchical models aim to capture this structured understanding computationally.

3.3.1 HIBERT: HIERACHICAL BIDIRECTIONAL ENCODER REPRESENTATIONS FROM TRANSFORMERS

HIBERT [3] introduced a two-level hierarchical architecture to explicitly model document structure, addressing the limitations of single-layer Transformers. Its design features a sentence-level encoder, a BERT-like Transformer that generates contextual sentence representations, and a document-level encoder, a second Transformer that models inter-sentence relationships from the sequence of sentence representations. This hierarchical structure allows HIBERT to process documents longer than BERT’s token limit while maintaining rich contextual understanding at both the sentence and document levels.

The model is pre-trained on unlabeled data using two main unsupervised strategies: masked sentence prediction, where entire sentences are masked and predicted from context, and additional document-level objectives that encourage the learning of document structure. For extractive summarization, HIBERT is fine-tuned as a sequence labeling model, where each sentence is assigned a binary label indicating its inclusion in the summary. This supervised fine-tuning leverages the rich hierarchical representations learned during pre-training.

3.3.2 STAS: SENTENCE-LEVEL TRANSFORMER BASED ATTENTIVE SUMMARIZATION

STAS [10] repurposes the pre-trained HIBERT model for purely unsupervised extractive summarization. Instead of fine-tuning, STAS leverages the internal mechanisms of the pre-trained model as ranking signals. The core innovation is to derive sentence importance scores directly from HIBERT’s sentence-level self-attention weights and its performance on the masked sentence prediction task. This approach allows STAS to rank sentences without any labeled data, making it highly adaptable. The pre-training of the underlying HIBERT model relies on objectives like masked sentence prediction, which teaches the model to understand sentence relationships and document context from unlabeled text.

3.3.3 HiSTRUCT+: HIERARCHICAL STRUCTURE-AWARE EXTRACTIVE SUMMARIZATION

HiStruct+ [43] enhances hierarchical Transformer architectures by explicitly encoding a document’s structural information—such as sections, subsections, and paragraphs—into the model’s

representations. It augments standard positional and segment embeddings with structure-aware signals, including section and paragraph indices, enabling the model to differentiate sentences across various discourse units. The architecture combines a token-level encoder for intra-sentence semantics with a sentence-level encoder that conditions on this structural metadata. This approach allows for structure-guided attention, promoting information flow within discourse neighborhoods while still capturing global context. Designed for long-document scalability, HiStruct+ uses efficient attention mechanisms to keep long inputs tractable. For summarization, a sentence classification head is trained to select summary sentences, leveraging the structure-aware features to balance salience and reduce redundancy across sections.

Taken together, hierarchical summarization has evolved from early “flat” encoders augmented with inter-sentence layers (e.g., BERTSUM’s inter-sentence Transformer as a first step toward sentence-level modeling [38]), to fully hierarchical encoders that separate sentence and document processing (HIBERT [3]), and further to models that make hierarchy explicit through structure-aware signals (HiStruct+ [43]). In parallel, efficient long-context Transformers (e.g., Longformer, BigBird) have reduced—but not eliminated—the need for explicit hierarchy by extending token windows [44, 45]. In this work we focus on HIBERT and STAS because they provide clear, sentence-level extractive scoring mechanisms grounded in hierarchical pretraining (masked sentence prediction) and attentive sentence interactions, fit our unsupervised extractive setting (STAS [10]), and allow direct comparison with a well-studied hierarchical baseline (HIBERT), without committing to additional structural annotations or heavy supervised fine-tuning required by some alternatives.

3.4 COMPARATIVE ANALYSIS OF ATS PARADIGMS

Table 3.1 provides a comprehensive comparison of the key approaches discussed, highlighting their evolution over time and different supervision requirements. The progression from statistical methods through deep learning to pre-trained language models reveals several important trends in the field.

From Surface-Level to Deep Semantic Understanding: The evolution from TF-IDF-based methods (TextRank, LexRank) to contextualized embeddings (PACSUM, GUSUM) represents a fundamental shift in how documents are represented. While early methods relied on word frequency statistics, modern approaches leverage deep contextual understanding from pre-trained models like BERT and Sentence-BERT. This transition has resulted in substantial improvements in summary quality, as demonstrated by the superior performance of

PACSUM and GUSUM over traditional graph-based methods.

The Supervision Dilemma: A critical observation from Table 3.1 is the trade-off between supervision requirements and practical applicability. Methods like BERTSUM and HIBERT achieve strong performance but require substantial labeled data for fine-tuning. In contrast, unsupervised methods like STAS, PMI-based approaches, and GUSUM maintain broader applicability across domains and languages where labeled data may be scarce or expensive to obtain.

Hierarchical vs. Flat Document Modeling: The comparison between flat sequence models (BERTSUM) and hierarchical architectures (HIBERT, HiStruct+) reveals the importance of explicit document structure modeling. Hierarchical methods demonstrate superior performance on long documents by separately processing sentence-level and document-level information, addressing the input length limitations that constrain flat models.

Table 3.2: Comparative Analysis of Text Summarization Paradigms

Paradigm	Strengths	Limitations	Applicability
Statistical Methods [21, 4, 23]	Unsupervised; Computationally efficient; Domain-agnostic.	Limited semantic understanding; Relies on surface-level features (e.g., word frequency).	Baselines; Resource-constrained environments; Quick information retrieval.
Deep Learning Models [34, 37, 36]	Improved semantic representation; End-to-end learning; Can model sequence information.	Requires large supervised datasets; Computationally expensive to train; Can struggle with very long documents.	Tasks where labeled data is abundant and high-quality summaries are needed.
Pre-trained Language Models [8, 38, 3]	Powerful contextual representations; Strong performance via transfer learning; Can be adapted for various tasks.	Fixed input length limitations; Often requires supervised fine-tuning; Can be computationally demanding.	Standard summarization tasks on moderate-length documents.

Table 3.1: Comprehensive Overview of Text Summarization Methods

Method	Year	Core Approach	Key Innovation	Supervision
TextRank[4]	2004	Graph centrality with TF-IDF	PageRank for sentence ranking	Unsupervised
LexRank[23]	2004	Graph centrality with cosine similarity	Eigenvector centrality	Unsupervised
SummaRuNNer[37]	2017	RNN-based sequence labeling	Sentence-level binary classification	Supervised
BERTSUM[38]	2019	BERT adaptation for summarization	Interval segment embeddings, inter-sentence Transformer layers	Supervised
PACSUM[39]	2019	Position-aware graph with BERT	Directional edges + BERT embeddings	Unsupervised
HIBERT[3]	2019	Hierarchical bidirectional Transformers	Document-level masked sentence prediction	Supervised
STAS[10]	2020	Hierarchical attention-based ranking	Leverages HIBERT's internal attention mechanisms	Unsupervised
PMI-based[41]	2021	Information-theoretic ranking	PMI for relevance/redundancy estimation	Unsupervised
RANKSUM[42]	2023	Multi-modal rank fusion	Ensemble of diverse features	Unsupervised
GUSUM[40]	2022	Graph-based with enhanced embeddings	Sentence-BERT + modified PageRank	Unsupervised

4

Methodology

4.1 EXTRACTIVE SUMMARIZATION ARCHITECTURE

All of the extractive summarization models follow a base general architecture, which is shown in Figure 4.1.[1]

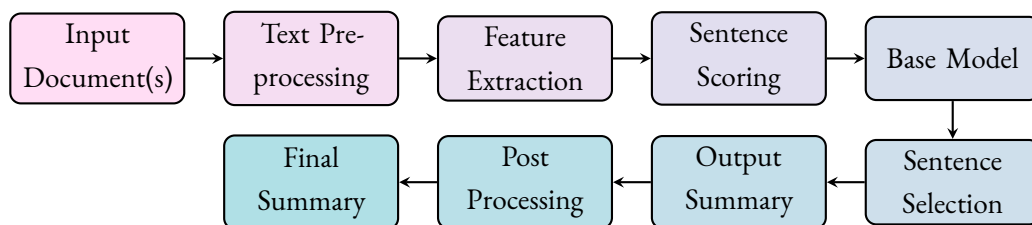


Figure 4.1: Architecture of an Extractive Text Summarization System.[1]

The general architecture of extractive summarization systems consists of several interconnected stages, each serving a specific purpose in transforming raw input documents into coherent summaries:

4.1.1 INPUT DOCUMENT(S)

The initial stage receives one or more source documents that require summarization. This component handles various input formats and document types, ranging from single news articles to

collections of related documents. The input stage is responsible for basic document validation and preparation for subsequent processing steps.

4.1.2 TEXT PRE-PROCESSING

This crucial preprocessing phase prepares the raw text for analysis by performing essential cleaning and normalization operations. Key activities include tokenization to break text into manageable units, sentence boundary detection to identify discrete textual segments, noise removal to eliminate irrelevant formatting elements, and standardization of text encoding. This stage ensures that downstream components receive consistently formatted and structured input.

4.1.3 FEATURE EXTRACTION

The feature extraction module transforms preprocessed text into numerical representations that capture semantic and syntactic properties. This stage employs various techniques such as term frequency analysis, part-of-speech tagging, named entity recognition, and semantic embedding generation. Modern approaches often utilize pre-trained language models to create rich contextual representations that capture deeper linguistic patterns and relationships within the text.

4.1.4 SENTENCE SCORING

Individual sentences are evaluated and assigned importance scores based on the extracted features. This component implements scoring algorithms that consider factors such as term significance, positional information, semantic relevance, and coherence with the overall document theme. The scoring mechanism determines which sentences are most likely to contribute valuable information to the final summary.

4.1.5 BASE MODEL

The core algorithmic component applies the primary summarization methodology, whether statistical, machine learning-based, or neural network-driven. This module integrates the scored sentences with the chosen summarization approach, implementing the fundamental logic that governs sentence selection decisions. Modern systems often employ transformer-based architectures or hierarchical neural networks at this stage.

4.1.6 SENTENCE SELECTION

Based on the base model’s output, this component makes final decisions about which sentences to include in the summary. The selection process considers factors such as summary length constraints, redundancy avoidance, and topical coverage. Advanced selection strategies may employ optimization techniques to ensure the chosen sentences provide comprehensive coverage while maintaining coherence.

4.1.7 OUTPUT SUMMARY

The selected sentences are compiled into an initial summary format. This stage performs basic aggregation of the chosen sentences, typically maintaining their original order or applying simple reordering rules. The output represents the core content that will form the final summary.

4.1.8 POST PROCESSING

The final refinement stage enhances the raw summary through various improvement techniques. Post-processing operations may include sentence reordering for better flow, pronoun resolution for clarity, redundancy elimination, coherence enhancement, and formatting standardization. This stage ensures the summary reads naturally and maintains logical progression.

4.1.9 FINAL SUMMARY

The ultimate output stage delivers the polished, user-ready summary that meets specified quality and length requirements. This component ensures the summary is properly formatted, coherent, and suitable for the intended application or user consumption.

4.2 INTRODUCTION TO THE PROPOSED METHOD

For our extractive summarization model, we chose a hierarchical approach that leverages both sentence-level and document-level representations. This allows the model to capture fine-grained details within sentences as well as a global understanding of the document structure. To do so we combine two state-of-the-art models: HIBERT [3] and STAS [10]. We take the pre-trained HIBERT model but use the ranking criteria of the STAS model. Later on we combined other ranking criteria as well such as metrics for improving the relevance and reducing the redundancy of the selected sentences [41], however our main ranking criteria is the STAS model’s one.

The motivation behind this combination is that, first, we chose HIBERT pretrained model over STAS pretrained model because we preferred the pretraining task of HIBERT model (masked sentence prediction) over the pretraining task of STAS model (sentence shuffling). They do sentence shuffling because they want the hierarchical document encoder to learn to select sentences based on their contents rather than positions, however usually in a document the positions of sentences have meaningful impact, for example in the articles the first sentences are usually more important. Second, we chose the ranking criteria of STAS model over HIBERT's one because however HIBERT model's training is unsupervised but it's extractive summarization is supervised, while STAS model's criteria is unsupervised.

4.3 BASE MODELS ARCHITECTURE

4.3.1 HIBERT MODEL

The Hierarchical Bidirectional Encoder Representations from Transformers (HIBERT) model [3] represents a supervised approach to extractive text summarization that utilizes a two-level hierarchical transformer architecture. This design enables the model to capture representations at both individual sentence and global document levels simultaneously.

For a given document $D = (S_1, S_2, \dots, S_{|D|})$ containing $|D|$ sentences, where each sentence $S_i = (w_1^i, w_2^i, \dots, w_{|S_i|}^i)$ consists of $|S_i|$ words, the model constructs hierarchical representations through a dual-encoder framework. The architecture incorporates a sentence-level encoder that processes individual sentences into vector representations, and a document-level encoder that models inter-sentence relationships within the broader document context. Both encoding components are built upon the transformer architecture [7], arranged in a nested hierarchical structure as illustrated in Figure 4.2. Each transformer layer consists of multi-head self-attention mechanisms followed by position-wise feed-forward networks, with residual connections [26] and layer normalization [27] applied throughout.

The model's pretraining consists of masked sentence prediction, since it aims to learn contextual representations by predicting masked tokens within sentences.

Document-Level Masking

The model randomly selects **15%** of sentences from document D for masking during pre-training. For each selected sentence, the masking procedure applies one of three operations:

- **80%** probability: Complete sentence masking by replacing all tokens with [MASK] symbols

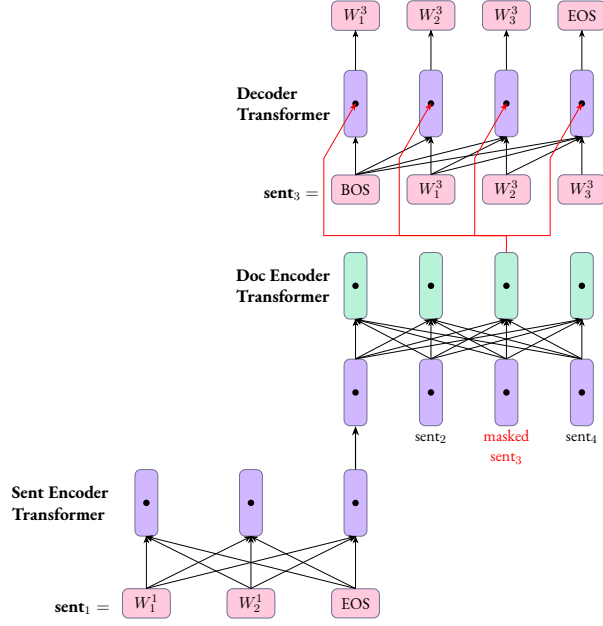


Figure 4.2: The architecture of HIBERT during training. sent_1 is a sentence in the document above, which has four sentences in total. sent_3 is masked during encoding and the decoder predicts the original sent_3 . [3]

- **10%** probability: Sentence preservation without any modifications
- **10%** probability: Sentence replacement with a randomly selected sentence from the corpus

Masked Sentence Prediction

Given the original document $D = (S_1, S_2, \dots, S_{|D|})$ and its masked version $\tilde{D} = (\tilde{S}_1, \tilde{S}_2, \dots, \tilde{S}_{|D|})$, the model learns to predict masked content through autoregressive generation. For a masked sentence S_k , the conditional probability of generating token w_j^k depends on previously generated tokens and the document context:

$$p(w_j^k | w_{0:j-1}^k, \tilde{D}) = \text{softmax}(\mathbf{W}^O \mathbf{g}_{j-1}) \quad (4.1)$$

where the parameters represent:

- $\tilde{g}_{j-1}$ denotes the hidden state at position $j - 1$, incorporating both self-attention over generated tokens and cross-attention to the document context
- $\mathbf{W}^O$ represents the learned output projection matrix that transforms hidden representations to vocabulary distributions

The complete objective maximizes the likelihood of all masked sentences $\mathcal{M}$ conditioned on the masked document context:

$$p(\mathcal{M}|\mathcal{D}) = \prod_{k \in \kappa} \prod_{j=1}^{|S_k|} p(w_j^k | w_{0:j-1}^k, \tilde{D}) \quad (4.2)$$

where κ indexes the set of masked sentences.

For extractive summarization tasks, HIBERT formulates the problem as a sequence labeling challenge where each sentence in the document receives a binary classification. The model treats documents as sequences of sentences and assigns binary labels (True/False) to indicate whether each sentence should be included in the extracted summary.

Consider a document $D = (S_1, S_2, \dots, S_{|D|})$ with corresponding binary labels $Y = (y_1, y_2, \dots, y_{|D|})$ derived through oracle extraction methods (detailed in Section 4.6). The model processes the document through its hierarchical transformer architecture to generate contextually enriched sentence representations $(d_1, d_2, \dots, d_{|D|})$, as shown in Figure 4.3. For each sentence S_i , the inclusion probability is computed through a learned linear transformation followed by softmax normalization:

$$p(y_i|D) = \text{softmax}(W_S d_i) \quad (4.3)$$

where $W_S \in \mathbb{R}^{2 \times d}$ represents the learned classification parameters. The model optimization minimizes the cross-entropy loss between predicted and oracle labels across all sentence-document pairs in the training corpus.

4.3.2 STAS MODEL

STAS (as shorthand for Sentence-level Transformer based Attentive Summarization) [10] is an unsupervised extractive summarization model. It employs the same hierarchical structure as HIBERT model [3]. Their main difference lies in their pretraining objectives and their summarization approach. The pretraining phase includes Masked Sentence Prediction task same as HIBERT model and a Sentence Shuffling task. In the sentence shuffling task, they randomly shuffle the order of sentences in a document and train the hierarchical encoder to select sentences based on their context, rather than their position.

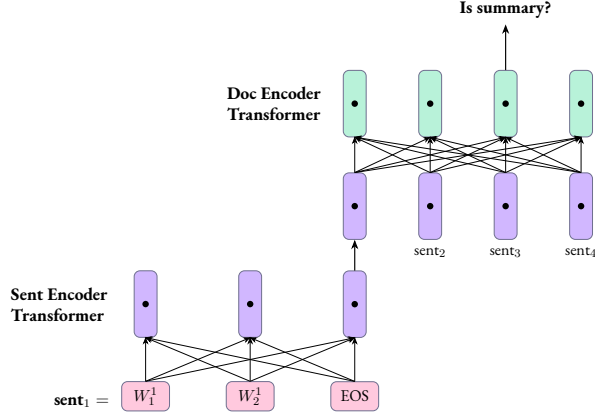


Figure 4.3: The architecture of HIBERT's extractive summarization model.[3]

4.3.2.1 STAS RANKING CRITERIA

STAS proposes an unsupervised two-stage ranking scheme. The first ranking criterion is based on sentence probabilities under a document model. Let $\mathcal{D} = (S_1, S_2, \dots, S_{|D|})$ be a document; its probability can be written as

$$p(\mathcal{D}) = \prod_{i=1}^{|D|} p(S_i \mid S_{1:i-1}) \approx \prod_{i=1}^{|D|} p(S_i \mid D_{\neg S_i}) \quad (4.4)$$

Directly estimating $p(S_i \mid S_{1:i-1})$ is difficult because the encoder is bidirectional. STAS therefore approximates it by $p(S_i \mid D_{\neg S_i})$, which is estimated using the masked-sentence prediction task (see Section 3.2). Concretely, after masking all tokens of S_i in $\mathcal{D}$, the sentence probability is computed token-wise and length-normalized. Denoting the words of S_i by $w_1^i, \dots, w_{|S_i|}^i$, STAS defines

$$\hat{r}_i = \frac{1}{|S_i|} \sum_{j=1}^{|S_i|} p(w_j^i \mid w_{0:j-1}^i, D_{\neg S_i}) \quad (4.5)$$

The authors then normalize these scores across sentences in the same document to obtain the first ranking criterion:

$$\tilde{r}_i = \frac{\hat{r}_i}{\sum_{j=1}^{|D|} \hat{r}_j} \quad (4.6)$$

The second ranking criterion models how other sentences contribute to the importance of a target sentence via the sentence-level self-attention matrix A produced by the Transformer encoder. Viewing the document as a directed graph where each sentence is a node and edge weights are given by attention, let $A_{j,i}$ denote the attention weight from sentence S_j to S_i . The contribution-aggregated score is

$$r'_i = \sum_{j=1, j \neq i}^{|D|} A_{j,i} \tilde{r}_j \quad (4.7)$$

The final ranking score combines the sentence’s own normalized probability and the contributions from other sentences:

$$r_i = \gamma_1 \tilde{r}_i + \gamma_2 r'_i \quad (4.8)$$

The coefficients γ_1, γ_2 are tuned on the development set. STAS computes r_i iteratively by assigning $r_i \leftarrow \tilde{r}_i$ and repeating the update of Equation (13) and the combination above for T iterations; the authors report that a small number of iterations (e.g. $T \leq 3$) works well.

STAS also optionally incorporates simple length and positional normalizations (length normalization is applied in $\hat{r}_i$ above) and recommends using the attention-derived propagation as an efficient way to capture inter-sentence influence without supervised labels [10].

4.3.3 RELEVANCE AND REDUNDANCY RANKING CRITERIA

We add two other ranking criteria to the STAS ranking criteria to improve the relevance and reduce the redundancy of the selected sentences based on pointwise mutual information [41]. The first additional criterion is based on the relevance of each sentence to the document, while the second criterion focuses on the redundancy among the selected sentences. Based on the article Unsupervised Extractive Summarization using Pointwise Mutual Information [41], we implement these criteria as follows:

4.3.3.1 RELEVANCE CRITERION

The relevance of a sentence s to a document d is calculated using pointwise mutual information (PMI). By definition, $\text{pmi}(s; d) = \log \frac{p(s,d)}{p(s)p(d)} = \log \frac{p(d|s)}{p(d)}$. In the original formulation, a language model p_{LM} is used to estimate the probabilities, with conditional probabilities computed by considering the condition sentence as the prefix:

$$\text{Rel}(s, d) \stackrel{\text{def}}{=} \text{pmi}(s; d) = \log \frac{p_{LM}(d|s)}{p_{LM}(d)} \quad (4.9)$$

This is consistent with the definition of relevance: seeing the summary, how well we can estimate the document content.

4.3.3.2 IMPLEMENTATION CONSIDERATIONS

While the theoretical formulation of PMI-based relevance and redundancy relies on language model probability estimates, practical implementation requires careful consideration of the available models. Although it might seem natural to use the HIBERT model for probability estimation, HIBERT’s bidirectional architecture and non-generative nature make it unsuitable for computing conditional probabilities of the form $p(d|s)$ or $p(s_j|s_i)$. Bidirectional models cannot provide the sequential probability estimates required for PMI calculations, as they process entire sequences simultaneously rather than generating tokens autoregressively.

To address this limitation, we adopt a token overlap approach for PMI estimation, which is a well-established method in text analysis and information retrieval. This approach approximates the probabilistic dependencies between sentences through surface-level token sharing, providing a computationally efficient alternative that has been shown to be effective in practice for summarization tasks.

4.3.3.3 REDUNDANCY CRITERION

Redundancy measures how much overlap exists among sentences in the context of summarization. While traditionally measured through semantic similarity, redundancy can also occur when one sentence is entailed by or highly overlaps with another, even if they express different meanings.

For our approach, we implement a position-aware redundancy measure that aligns with the observation that early sentences in documents are typically more informative. This design choice is motivated by the effectiveness of the LEAD-3 baseline, which demonstrates that the first few sentences in news articles often contain the most crucial information.

Rather than measuring redundancy against already selected summary sentences, we compute redundancy with respect to sentences that precede the candidate sentence in the document order. This strategy serves two purposes: (1) it preserves the importance of early sentences

by penalizing later sentences that are redundant with them, and (2) it amplifies the salience of early sentences by assigning them lower redundancy scores.

Formally, for a candidate sentence s_i at position i in document D , we define:

$$\text{Red}(s_i, D) = \sum_{j=1}^{i-1} \text{pmi}(s_i; s_j) \quad (4.10)$$

where the summation is over all sentences s_j that appear before s_i in the document. This formulation ensures that sentences appearing later in the document are penalized if they contain information already present in earlier sentences, thereby maintaining the natural importance hierarchy observed in well-structured documents.

4.3.3.4 SENTENCE EXTRACTION

Given the relevance and position-aware redundancy criteria defined above, we aim to select important sentences from the document that maximize relevance while minimizing redundancy with respect to earlier sentences. These criteria will be combined with the STAS ranking criteria in our proposed architecture to create a comprehensive scoring mechanism for extractive summarization.

4.4 PROPOSED COMBINED ARCHITECTURE

Our proposed approach combines the hierarchical document representations from HIBERT with the unsupervised ranking criteria from STAS, enhanced by PMI-based relevance and redundancy measures. The architecture leverages four distinct ranking criteria to create a comprehensive scoring mechanism for sentence selection.

4.4.1 COMBINED RANKING STRATEGY

We integrate four ranking criteria to select the most informative and non-redundant sentences: **STAS Normalized Probability** ($\tilde{r}_i$) representing sentence probability normalized across the document, **STAS Attention-based Score** (r'_i) capturing contribution-aggregated scores from other sentences via attention, **PMI-based Relevance** ($\text{Rel}(s_i, D)$) measuring sentence relevance to the entire document, and **PMI-based Redundancy** ($\text{Red}(s_i, S)$) quantifying redundancy with already selected sentences. The final scoring function combines these four criteria

with learnable weights:

$$\text{Score}(s_i, S, D) = \alpha_1 \tilde{r}_i + \alpha_2 r'_i + \alpha_3 \text{Rel}(s_i, D) + \alpha_4 \text{Red}(s_i, S) \quad (4.11)$$

where S represents the selected summary sentences, D is the document, and $\alpha_1, \alpha_2, \alpha_3, \alpha_4$ are tunable coefficients that balance the contribution of each criterion. Note that the redundancy term can be either positive or negative depending on the desired effect: positive α_4 encourages selection of sentences that are coherent with already selected content, while negative α_4 promotes diversity by penalizing redundant sentences.

4.4.2 SENTENCE SELECTION ALGORITHM

Our extractive summarization process follows a greedy selection strategy that maximizes the combined scoring function. The complete algorithm is presented in Algorithm 4.1, which integrates all four ranking criteria within an iterative selection framework.

Our algorithm implements a multi-criteria approach that combines four distinct scoring mechanisms to achieve comprehensive sentence evaluation for extractive summarization. The process begins by computing reconstruction scores through masked sentence prediction using the pre-trained HIBERT model, which captures each sentence’s importance based on its predictability within the document context. Simultaneously, attention scores are derived by averaging multi-head attention weights to model inter-sentence dependencies. The algorithm then incorporates PMI-based relevance scores to measure how well each sentence represents the overall document content, and PMI-based redundancy scores to quantify information overlap with preceding sentences, thereby preserving the natural importance hierarchy of document structure. All scores undergo document-level min-max normalization to ensure fair comparison across different documents and score types. Finally, the algorithm performs systematic weight optimization by evaluating all possible combinations of the four scoring criteria, enabling automatic discovery of the optimal balance between content importance, contextual relationships, document relevance, and information diversity for each specific summarization task.

4.4.3 HYPERPARAMETER TUNING

To determine the optimal values for the coefficients $\alpha_1, \alpha_2, \alpha_3, \alpha_4$ in our combined scoring function, we performed a systematic parameter search over the validation set. This hyperparam-

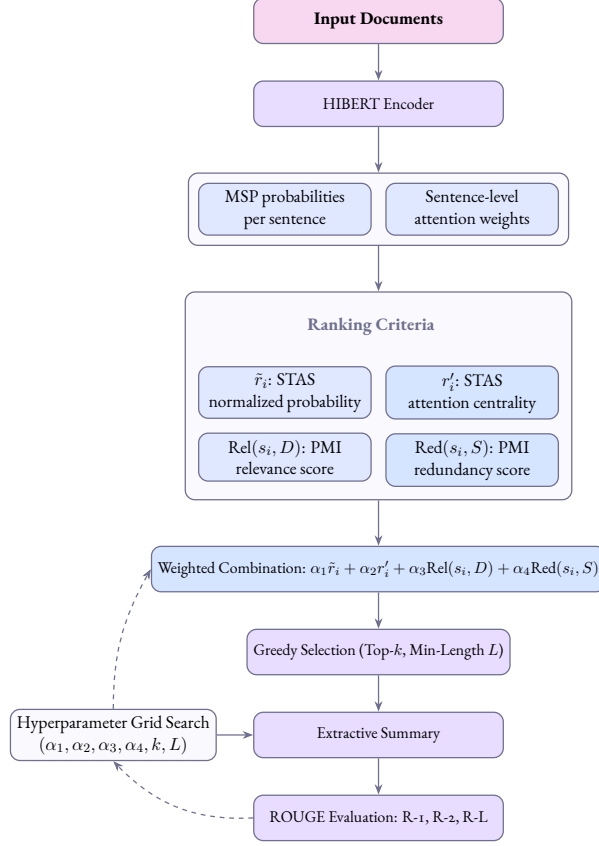


Figure 4.4: Architecture of the proposed combined summarization model. After getting the masked sentence prediction (MSP) probabilities and attention weights from the HIBERT encoder, we compute multiple ranking criteria for each sentence. STAS normalized probability, STAS attention centrality, PMI relevance, and PMI redundancy approximated with token overlap. These are combined using learned weights and fed to a greedy selection algorithm for extractive summarization.

eter optimization process aimed to find the coefficient combination that maximizes ROUGE scores while ensuring the model generalizes well to unseen data.

We conducted a grid search over predefined ranges for each coefficient:

- α_1 (STAS normalized probability): $[0.0, 0.2, 0.4, 0.6, 0.8, 1.0]$
- α_2 (STAS attention-based score): $[0.0, 0.2, 0.4, 0.6, 0.8, 1.0]$
- α_3 (PMI relevance): $[0.0, 0.2, 0.4, 0.6, 0.8, 1.0]$
- α_4 (PMI redundancy): $[0.0, 0.2, 0.4, 0.6, 0.8, 1.0]$

For each parameter combination, we evaluated the model performance on the validation set using ROUGE-1, ROUGE-2, and ROUGE-L metrics.

Algorithm 4.1 Combined Multi-Criteria Extractive Summarization

Require: Document $D = \{S_1, S_2, \dots, S_n\}$, Summary length k , Length threshold L

Ensure: Selected sentence indices for optimal weight combination

```
1: Compute Reconstruction Scores:
2: for each sentence  $S_i$  in  $D$ 
3:   Mask  $S_i$ , perform forward pass, compute reconstruction probability
4: end for
5: Compute Attention Scores:
6: Extract and average attention weights across heads/layers
7: Compute STAS Scores:
8: Normalize reconstruction scores:  $\tilde{r}_i = \frac{\hat{r}_i}{\sum_{j=1}^{|D|} \hat{r}_j}$ 
9: Compute attention-weighted scores:  $r'_i = \sum_{j=1, j \neq i}^{|D|} A_{j,i} \tilde{r}_j$ 
10: Compute PMI Scores:
11: for each sentence  $S_i$ 
12:   Calculate relevance using token coverage with document
13:   Calculate redundancy with preceding sentences
14: end for
15: Normalize and Filter:
16: Apply min-max normalization per document to all score types
17: Filter sentences below length threshold  $L$ 
18: Weight Optimization and Selection:
19: for each weight combination  $(\alpha_1, \alpha_2, \alpha_3, \alpha_4)$ 
20:   Compute:  $\text{Score}(S_i) = \alpha_1 \cdot \text{recon}_i + \alpha_2 \cdot \text{attn}_i + \alpha_3 \cdot \text{rel}_i + \alpha_4 \cdot \text{red}_i$ 
21:   Select top- $k$  sentences based on combined scores
22: end for
23: return Optimal sentence selection across all weight combinations
```

The hyperparameter tuning process follows a rigorous validation protocol to prevent overfitting:

1. All parameter combinations are evaluated exclusively on the validation set
2. The optimal coefficient set is selected based on the highest ROUGE-L score
3. The selected hyperparameters are then applied to the test set for final evaluation

4.5 TECHNICAL IMPLEMENTATION

4.5.1 HIBERT ARCHITECTURE

Our implementation utilizes the pre-trained HIBERT model as the foundation for hierarchical document representation. Considering L as the number of layers in Transformer, H the hidden size and A the number of attention heads; the original HIBERT work [3] introduced two model variants: HIBERT_S ($L = 6$, $H = 512$, $A = 8$) and HIBERT_M ($L = 6$, $H = 768$, $A = 12$). We chose HIBERT_M for our implementation due to its superior representational capacity while maintaining computational efficiency.

Parameter	Value
Vocabulary	32,770 tokens
Hidden Size	768
Encoder Layers	6
Attention Heads	12
FFN Size	3,072
Layer Norm ϵ	1e-05
Position Embeddings	Sinusoidal

Table 4.1: HIBERT_M architecture specifications

The original HIBERT_M model was trained using distributed computing across 8 NVIDIA Tesla V100 GPUs with a batch size of 256 documents, employing Adam optimizer with learning rate of $1e-4$, $\beta_1 = 0.9$, $\beta_2 = 0.999$, L2 weight decay of 0.01, and dropout rate of 0.1 across all layers. By leveraging this pre-trained model, our approach benefits from robust hierarchical representations while focusing on developing effective ranking mechanisms for extractive summarization. The model has approximately 110 million parameters, with around 25 million in the embedding layer, 85 million in the encoder layers.

4.5.2 FAIRSEQ LIBRARY

All the programming work in our project is based on the Fairseq library [46]. Since both the HIBERT project and the STAS reference implementation are built on top of Fairseq [47, 48], we likewise adopt Fairseq to run our models. Fairseq is a sequence modeling toolkit written in PyTorch by Meta AI. It provides a modular, registry-based design where components are selected at runtime via command-line arguments. Every experiment is defined by five core choices: The

framework consists of **Tasks** that handle data operations and dataset loading, **Models** that define the neural network architecture and forward pass, **Criteria**s that compute loss functions, **Optimizers** that update model parameters based on gradients, and **Learning Rate Schedulers** that dynamically adjust learning rates during training either after each parameter update or at epoch boundaries. [49]

4.5.3 PREPROCESSING PIPELINE

The preprocessing pipeline for the CNN/Daily Mail dataset involved several key steps to ensure the data was in a suitable format for training and evaluation:

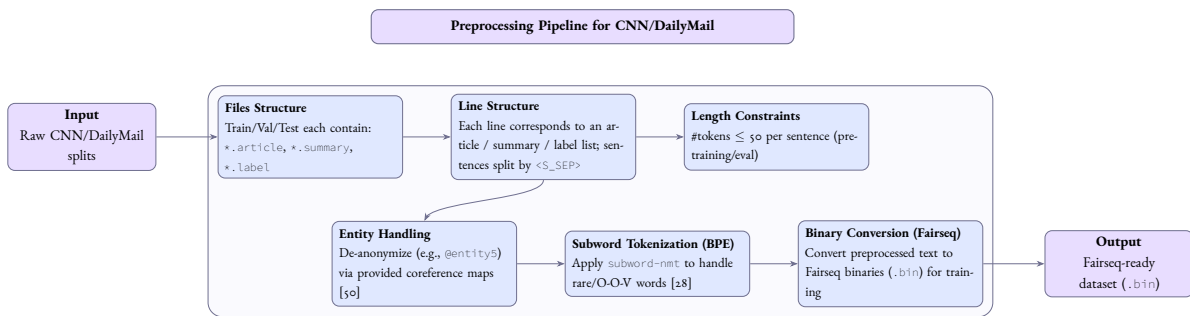


Figure 4.5: Preprocessing Pipeline Applied to the CNN/DailyMail Dataset.

1. **Files Structure:** The training, validation and test splits should contain the `*.article`, `*.summary` and `*.label` files.
2. **Line Structure:** Each line in `*.article`, `*.summary` and `*.label` is consequently an article, a summary, and gold sentence level label list of an article and sentences should be separated by `<S_SEP>` tokens.
3. **Length Constraint** The max number of tokens per sentence is set to 50 during pretraining and evaluation.
4. **Entity Handling:** Replaced anonymized entities (e.g., `@entity5`) with original referents using provided coreference maps [50].
5. **Byte-Pair Encoding (BPE):** Byte-Pair Encoding (BPE) [28] should be applied using `subword-nmt` library, to handle rare and out-of-vocabulary words by splitting them into more frequent subword units.
6. **Binary Format Converting:** The text should be converted to Binary Format to transform it into efficient Fairseq storage (`.bin`) for training.

4.6 ORACLE EXTRACTION

The oracle extraction method we used is the same as [37] article. It provides a theoretical upper bound for extractive summarization performance by selecting the optimal subset of sentences that maximizes the similarity with the reference summary. This approach is crucial for understanding the potential of extractive methods and serves as a benchmark for evaluating summarization models.

The greedy oracle selection process works as follows: given a document $D = \{s_1, s_2, \dots, s_n\}$ with n sentences and a reference summary R , the algorithm aims to find a subset $S^* \subseteq D$ that maximizes the ROUGE score with respect to R .

Algorithm 4.2 SummaRuNNer Greedy Oracle Algorithm

Require: Document $D = \{s_1, s_2, \dots, s_n\}$, Reference summary R , Maximum summary length L

Ensure: Oracle summary S^*

```
1: Initialize  $S^* = \emptyset$  (empty summary)
2: Initialize remaining =  $D$  (remaining sentences)
3: while  $|S^*| < L$  and remaining  $\neq \emptyset$ 
4:   best_sentence = null
5:   best_score =  $-\infty$ 
6:   for each sentence  $s_i \in$  remaining
7:     candidate_summary =  $S^* \cup \{s_i\}$ 
8:     score = ROUGE(candidate_summary,  $R$ )
9:     if score > best_score
10:       best_score = score
11:       best_sentence =  $s_i$ 
12:   end if
13: end for
14: if best_sentence  $\neq$  null
15:    $S^* = S^* \cup \{\text{best\_sentence}\}$ 
16:   remaining = remaining  $\setminus \{\text{best\_sentence}\}$ 
17: else
18:   break {No improvement possible}
19: end if
20: end while
21: return  $S^*$ 
```

This greedy approach ensures that each selected sentence contributes maximally to the over-

all ROUGE score improvement. While not guaranteed to find the globally optimal solution, the greedy oracle provides a practical and effective upper bound for extractive summarization systems. The algorithm’s computational complexity is $O(n^2 \cdot L)$, where n is the number of sentences in the document and L is the maximum summary length.

4.7 EVALUATION METRICS

The main evaluation metrics we used are ROUGE-1, ROUGE-2, and ROUGE-L [17]. ROUGE (Recall-Oriented Understudy for Gisting Evaluation) is a set of metrics commonly used to evaluate the quality of summaries by comparing them to reference summaries.

- ROUGE-1: Measures the overlap of unigrams between the generated summary and the reference summary.
- ROUGE-2: Measures the overlap of bigrams between the generated summary and the reference summary.
- ROUGE-L: Measures the longest common subsequence between the generated summary and the reference summary.

5

Experimental Results

5.1 DATASET DESCRIPTION

We use **CNN/Daily Mail dataset** [11], for our evaluation. CNN/Daily Mail is one of the most widely-used dataset for summarization. It contains news articles and associated highlights as summaries written by journalists from CNN and the DailyMail. [1]

The dataset comprises three essential components:

- **ID:** A hexadecimal string representing the unique URL identifier from which the story was originally retrieved.
- **Article:** The complete textual content of the news article.
- **Highlights:** The article summary composed by humans, serving as the reference summary for evaluation.

The mean token counts for articles and highlights in the dataset are shown in Table 5.1. The dataset is split into training, validation, and test sets, with statistics provided in Table 5.2. We use the non-anonymized version of the dataset, which retains the original named entities and all the preprocessing steps are shown in Figure 4.5. It is also worth noting that the vocabulary size of this dataset consists of 32,770 tokens.

Feature	Mean Token Count
Article	781
Highlights	56

Table 5.1: Mean Token Count by Feature in CNN/DailyMail Dataset

Dataset Split	Number of Instances
Train	287, 226
Validation	13, 368
Test	11, 490

Table 5.2: Dataset Split Statistics for CNN/DailyMail

5.2 EXPERIMENTAL SETUP

5.2.1 HARDWARE CONFIGURATION

All experiments were conducted on cloud-based GPU infrastructure provided by Lightning AI [51]. The hardware configuration varied based on the computational requirements of different experimental phases:

- **NVIDIA L40S GPU:** Used for processing the complete test set and performing comprehensive document ranking operations. This high-performance GPU provided the necessary computational power for handling large-scale inference tasks across the entire CNN/DailyMail test dataset.
- **NVIDIA Tesla T4 GPU:** Employed for smaller test subsets and ROUGE metric calculations. This configuration offered sufficient computational resources for evaluation tasks while maintaining cost efficiency.

5.2.2 MODEL CONFIGURATION

For our extractive summarization approach, we utilize the HIBERT_M model variant as our base hierarchical encoder. Following the original HIBERT architecture specifications [3], HIBERT_M

is configured with $L=6$ transformer encoder layers, $H=768$ hidden dimensions, and $A=12$ attention heads. This configuration provides superior representational capacity compared to the smaller HIBERT_S variant while maintaining computational efficiency for our task requirements.

We employ the pre-trained HIBERT_M checkpoint (checkpoint100.pt) which was trained on a large-scale corpus for the masked sentence prediction task. This checkpoint serves as the foundation for our hierarchical document representations, providing robust sentence-level and document-level encodings that we subsequently combine with STAS ranking criteria for unsupervised extractive summarization.

5.2.3 INFERENCE PARAMETERS

To adapt the HIBERT model for our extractive summarization task, we modified its processing strategy from the original pre-training approach. During pre-training, HIBERT used random sampling with batch size 8, where in each batch, 5 sentences were randomly masked while 3 sentences remained unmasked for prediction.

However, for our extractive summarization task using STAS ranking criteria, we needed to obtain probability scores for all sentences in the document. This required two key modifications:

1. **Sequential Processing:** We adopted a sequential, sentence-by-sentence processing method with batch size 1. For each document, we create a separate batch for each sentence, where only one sentence is masked while all other sentences are unmasked. This allows us to generate individual probability scores for every sentence.
2. **Ordered Processing:** We removed the sentence shuffling mechanism used in HIBERT’s original training to maintain the original sentence order. This ensures we can correctly map the probability scores back to their corresponding sentences in the document.

This sequential processing approach is crucial for our extractive summarization task, as it enables us to score each sentence independently and maintain proper sentence indexing before applying the STAS ranking criteria to select the most salient sentences for the summary.

It is important to distinguish between two token length parameters used in our approach. The **Max Sentence Token** parameter (50 tokens) is applied during the preprocessing and inference stages for memory management purposes, where sentences exceeding this length are shortened to ensure efficient memory utilization. In contrast, the **Min Summary Sentence**

Token parameter (20 tokens) is applied during the sentence selection phase for summary generation. While all sentences in the document are processed and scored regardless of their length, during the final ranking and selection process, sentences shorter than 20 tokens are excluded from consideration as they typically lack sufficient informative content for meaningful summarization. This two-stage filtering approach ensures computational efficiency while maintaining the quality of the selected summary sentences. Top 3 sentences are selected based on the highest probability scores, provided they meet the minimum length requirement.

Parameter	Value
Min Summary Sentence Token(L)	20
Max Sentence Token	50
Batch Size	1
Shuffling	False
Top-K (Selected Sentences)	3

Table 5.3: Inference Parameters Configuration

5.3 BASELINE COMPARISONS

We evaluate our proposed method using ROUGE-1, ROUGE-2, and ROUGE-L metrics against two key benchmarks: oracle summaries and state-of-the-art extractive summarization models.

5.3.1 ORACLE SUMMARIES

Oracle summaries represent the upper bound for extractive summarization performance on the CNN/DailyMail dataset. These are created by selecting the combination of sentences from each article that maximizes ROUGE scores against the reference summary. Comparing our method against oracle performance indicates how close our unsupervised approach comes to the theoretical maximum achievable by any extractive method.

5.3.2 STATE-OF-THE-ART MODELS

We compare our method against several recent extractive summarization systems, encompassing both traditional unsupervised methods and more recent neural approaches. This compar-

ison helps to contextualize the performance of our model. For the convinience of the reader, brief explanation of these models are written bellow. The key baselines include:

- **LEAD-3**: A widely-used and robust baseline that constructs a summary by simply selecting the first three sentences of the source document.
- **TEXTRANK** [4]: A graph-based ranking model that adapts the PageRank algorithm to sentence graphs, where sentences are nodes and edges are weighted by lexical similarity.
- **LEXRANK** [23]: Similar to TextRank, this graph-based method uses eigenvector centrality on a cosine similarity graph to identify the most central sentences.
- **PACSUM** [39]: A position-aware graph-based model that uses BERT embeddings and incorporates positional bias to rank sentences.
- **GUSUM** [40]: A graph-based unsupervised summarization model that uses Sentence-BERT embeddings and a modified PageRank algorithm to identify salient sentences.
- **PMI-based** [41]: An information-theoretic approach that uses a pre-trained language model to estimate sentence relevance and redundancy based on Pointwise Mutual Information (PMI).
- **STAS** [10]: An unsupervised model that repurposes a pre-trained HIBERT model, using its internal self-attention weights and masked sentence prediction to rank and select sentences.
- **BERTSUM** [38]: A supervised model that adapts the BERT architecture for extractive summarization.
- **HIBERT** [3]: A supervised hierarchical method, that has a token-level transformer and a sentence-level transformer to learn interactions between sentences with self-attention. We compare against both the small (HIBERT_S) and medium (HIBERT_M) variants.

5.4 MAIN RESULTS

Table 5.4 presents the performance comparison of our proposed method against established baseline models on the CNN/DailyMail test set. Our approach combines HIBERT hierarchical representations with STAS ranking criteria and PMI based criteria for relevance and redundancy. The algorithm is detailed in Algorithm 4.1.

The sentence scores based on Algorithm 4.1 are calculated as follows:

$$\text{score} = \alpha_1 \tilde{r}_i + \alpha_2 r'_i + \alpha_3 \text{Rel}(s_i, D) + \alpha_4 \text{Red}(s_i, S) \quad (5.1)$$

For easier notation, let the four criteria be r_1, r_2, r_3 , and r_4 respectively. The first two criteria are from the STAS model explained in details in Methodology Chapter 4, while the last two are based on PMI for relevance and redundancy. $\alpha_1, \alpha_2, \alpha_3, \alpha_4$ are the weights for each criterion. Each one is selected from $[0, 1]$ with a step size of 0.2, and the best combination is found using a grid search on the validation set using ROUGE metrics. Total number of possible combinations are 56, and the best combination is $\alpha_1 = 0.2, \alpha_2 = 0.2, \alpha_3 = 0.0, \alpha_4 = 0.6$.

Method	ROUGE-1	ROUGE-2	ROUGE-L
Unsupervised			
LEAD-3	40.34	17.70	36.57
TEXTRANK (tf-idf) [4]	33.20	11.80	29.60
LEXRANK [23]	34.68	12.82	31.12
PACSUM [39]	40.70	17.80	36.90
GUSUM [40]	43.40	17.02	42.38
PMI-based [41]	36.68	14.52	23.32
STAS [10]	40.90	18.02	37.21
Supervised			
BertSum [38]	43.25	20.24	39.63
HIBERT _S [3]	42.31	19.87	38.78
HIBERT _M [3]	42.37	19.95	38.83
Oracle(1 to 3 sentences)	54.55	32.42	51.74
Oracle(3 sentences)	53.95	31.77	51.21
Ours	41.06	16.15	33.90

Table 5.4: Performance comparison on CNN/DailyMail test set using ROUGE-1, ROUGE-2, and ROUGE-L metrics.

As we can see in Table 5.4, our proposed method is strongly comparable with state-of-the-art unsupervised models, and outperforms most of them in ROUGE-1 and also performs much better in ROUGE-2 and ROUGE-L compared to basic models while being competitive with more advanced unsupervised models.

Compared to supervised models, our method is slightly behind, but it is expected since su-

pervised models are trained on the specific dataset and task, while our method is unsupervised and does not require any training on the specific dataset. However, the larger gap compared to the oracle summaries, which are the upper bound for extractive summarization, shows that the differences between supervised and unsupervised methods are not as significant when compared to the theoretical maximum performance.

We also present the results of some other variations of our model, by using different methods for our last two criteria, in Table 5.5. For these variations, we test two common similarity metrics: **N-gram Overlap**, which measures similarity based on the proportion of shared word sequences (n-grams) between sentences, and **Cosine Similarity**, which measures the semantic similarity between sentence embeddings generated by Sentence-BERT. Since the relevance based on the PMI was insignificant in the grid search, and our best result was achieved with $\alpha_3 = 0$, we first compare the results of our base model with variants that use only the redundancy criterion (r_4) plus the STAS criteria.

Model Variant	Weights	ROUGE-1	ROUGE-2	ROUGE-L
Base Model(Ours)	(0.2,0.2,0.0,0.6)	41.06	16.15	33.90
N-gram Overlap (n=2)	(0.2,0.2,0.0,0.6)	33.70	10.65	28.03
N-gram Overlap (n=3)	(0.0,0.2,0.0,0.8)	35.22	11.75	29.66
N-gram Overlap (n=4)	(0.0,0.4,0.0,0.6)	35.34	11.76	29.59
Cosine Similarity*	(0.0,0.4,0.0,0.6)	36.57	12.77	30.56

Table 5.5: Performance comparison of model variants using different redundancy criteria on CNN/DailyMail test set.

*Cosine similarity with sentence-BERT embeddings.

As we can see, using the PMI-based redundancy criterion (r_4) based on token overlap, achieves significantly higher scores compared to using n-gram overlap or cosine similarity with sentence-BERT embeddings. This shows we have a good method for capturing redundancy, but we can also experiment to see if there are better methods for capturing the relevance. Thus we first evaluate our performance using only the PMI-based relevance (r_3) and redundancy (r_4) criteria, and then we compare it to other variants that also use only the relevance criterion and the redundancy criterion, but with different methods for calculating relevance and redundancy.

In Table 5.6, we present the results of these variants. We have also conducted grid search for the weights of the relevance and redundancy criteria, for the other variants as well, to make sure we are getting the best possible performance across variations. We can see that the results of using only n-gram for relevance and redundancy, does not lead to good results. However, by using only the cosine similarity based relevance and redundancy criteria, we can achieve signif-

Model Variant	Weights	ROUGE-1	ROUGE-2	ROUGE-L
Ours (Base)	(0.2,0.2,0.0,0.6)	41.06	16.15	33.90
PMI	(0.0,0.0,0.4, 0.6)	36.83	13.11	30.03
Cosine Similarity	(0.0,0.0,0.6,0.4)	38.45	13.75	31.70
Cosine Similarity	(0.0,0.0,0.4,0.6)	39.22	14.63	32.55
Cosine Similarity	(0.0,0.0,0.2,0.8)	37.52	13.47	31.21
N-gram Overlap (n=2)	(0.0,0.0,0.2,0.8)	30.30	8.31	25.22
N-gram Overlap (n=3)	(0.0,0.0,0.2,0.8)	31.15	9.10	26.05
N-gram Overlap (n=4)	(0.0,0.0,0.2,0.8)	30.50	8.62	25.43

Table 5.6: Performance comparison of model variants using different relevance and redundancy criteria combinations on CNN/DailyMail test set.

icantly higher scores compared to using only the cosine similarity based redundancy criterion plus r_1 or r_2 in Table 5.5. Thus we can extrapolate that using the cosine similarity based relevance criterion plus the PMI-based redundancy criterion, must achieve even higher scores.

This is indeed the case, compared to using only the PMI-based or cosine similarity based relevance and redundancy criteria, as shown in Table 5.7, but it still does not achieve the scores of our base model. This may be due to the fact that even though the cosine similarity based redundancy score is effective while used with the cosine similarity based relevance score, the PMI-based redundancy score is more effective while used with the STAS relevance scores (r_1 and r_2). That shows our criteria are complementary, and using the right combination of them is important to achieve the best performance.

Model Variant	Weights	ROUGE-1	ROUGE-2	ROUGE-L
Ours (Base)	(0.2,0.2,0.0,0.6)	41.06	16.15	33.90
PMI Only	(0.0,0.0,0.4,0.6)	36.83	13.11	30.03
Cosine Similarity (r_3, r_4)	(0.0,0.0,0.4,0.6)	39.22	14.63	32.55
Cosine Similarity(r_3), PMI(r_4)	(0.0,0.0,0.6,0.4)	39.00	14.20	31.81
Cosine Similarity(r_3), PMI(r_4)	(0.0,0.0,0.4,0.6)	40.10	15.14	33.07
Cosine Similarity(r_3), PMI(r_4)	(0.0,0.0,0.2,0.8)	40.07	14.77	32.81

Table 5.7: Component-wise comparison of relevance and redundancy criteria on CNN/DailyMail. Mixing cosine-based relevance (r_3) with PMI-based redundancy (r_4) improves over PMI-only and cosine-only (r_3, r_4) setups, but remains below the base model that combines STAS relevance (r_1, r_2) with PMI redundancy (r_4).

Now that we are sure that using the PMI-based redundancy criterion is an effective way to enhance the performance of our STAS based criteria, we can compare the results of our model

while using the PMI-based redundancy criterion, with the results of using only the STAS based criteria, as shown in Table 5.8. We can see that by adding the PMI-based redundancy criterion, we can achieve significantly higher scores compared to using only r_1 and r_2 .

Model Variant	ROUGE-1	ROUGE-2	ROUGE-L
HIBERT using STAS criteria	35.41	11.94	29.48
Ours with Redundancy control	41.06	16.15	33.90

Table 5.8: ROUGE scores of HIBERT with STAS criteria, and HIBERT with STAS plus Redundancy criteria on the CNN/DailyMail test set.

5.5 DISCUSSION

The experimental results demonstrate that our proposed method achieves competitive performance in the unsupervised extractive summarization domain while revealing important insights about the complementary nature of different ranking criteria. Our approach, which combines HIBERT hierarchical representations with STAS ranking criteria and PMI-based redundancy control, achieves ROUGE scores of 41.06, 16.15, 33.90 (R_1 , R_2 , R_L), positioning it competitively among state-of-the-art unsupervised methods. Notably, our method outperforms simpler traditional approaches like TEXTRANK and LEXRANK by substantial margins, while remaining competitive with more recent methods such as STAS and PACSUM. The performance gap with supervised methods like BertSum is relatively modest, which is expected given that supervised approaches benefit from task-specific training on the target dataset.

A critical finding from our ablation studies is the effect of combining different ranking criteria. The analysis in Tables 5.8 and 5.7 reveals that using HIBERT with only STAS criteria achieves lower ROUGE scores, while incorporating PMI-based redundancy control substantially improves performance to our final scores. This improvement in ROUGE demonstrates that redundancy control is essential for effective extractive summarization. Interestingly, our grid search optimization revealed that the PMI-based relevance criterion (r_3) contributed minimally ($\alpha_3 = 0.0$ in the optimal configuration), suggesting that the STAS criteria already capture document relevance effectively, while the primary benefit comes from improved redundancy management through PMI-based token overlap analysis.

The comparison of different redundancy calculation methods (Table 5.5) further validates our approach. PMI-based redundancy significantly outperforms alternatives such as n-gram

overlap and cosine similarity with sentence-BERT embeddings. This superiority can be attributed to PMI’s ability to capture meaningful statistical dependencies between tokens. The poor performance of n-gram overlap methods highlights the limitation of purely syntactic similarity measures, while cosine similarity approaches, though more complex, still performs worse than PMI method.

However, our results also reveal opportunities for improvement. The substantial gap between our method and oracle performance indicates significant room for advancement in unsupervised extractive summarization. Additionally, while our approach excels in ROUGE–1 scores, the performance in ROUGE–2 suggests that capturing bi-gram level coherence remains challenging.

The qualitative example in Table 5.9 illustrates an example of our model generating a summary. The italic sentences indicate sentences that were written twice in the main article, and the highlighted sentences are those selected by our model. As we can see, our model is able to understand that these sentences are redundant and only select one of them. Also, we can see that our model is able to select relevant sentences that cover the main points of the article, but it also selects some less relevant sentences. This indicates that while our ranking criteria are effective, there is still room for improvement in accurately identifying the most salient content.

Summary Type	Content
Gold Summary	Lippert, 42, suffered deep gashes to his hand when he was assaulted by knife-wielding nationalist Kim Ki-Jong in central Seoul last month. 'It is an amazing apparatus, one I haven't seen before - so innovative and creative,' he wrote on Facebook.
Ours	South Korea Ambassador Lippert is now wearing a dynamic exoskeletal brace on his hand while he recovers from last month's knife attack. Mark Lippert, 42, suffered deep gashes to his cheek and hand when he was assaulted by knife-wielding nationalist Kim Ki-Jong at a breakfast function in central Seoul last month. The envoy required 80 stitches to the wound on his face and two-and-a-half hours of surgery. Scroll down for video. <i>South Korea Ambassador Lippert is now wearing a dynamic exoskeletal brace on his hand (pictured) while he recovers from last month's knife attack. Mark Lippert, 42, suffered deep gashes to his cheek and hand when he was assaulted by knife-wielding nationalist Kim Ki-Jong at a breakfast function in central Seoul last month.</i> Pictured: Lippert pictured with the brace with his wife and young baby. The brace was created by Seoul's Severance Hospital and Lippert marveled at how quickly they had constructed it. 'It is an amazing apparatus, one I haven't seen before - so innovative and creative,' he wrote on Facebook. 'It is very comfortable and has made a huge difference in my recovery as I am able to hold things in my hand.

Table 5.9: Qualitative comparison of a generated summary against the gold summary. Highlighted sentences are those selected by our model. The italic text indicates sentences that were written twice in the main article.

6

Conclusion

This thesis explored the application of hierarchical transformer architectures to unsupervised extractive text summarization. We developed a novel approach that integrates HIBERT’s hierarchical document representations with STAS ranking criteria, incorporating a PMI-based redundancy control mechanism for enhanced performance. The results demonstrate that hierarchical modeling, when combined with principled unsupervised ranking techniques, provides an effective solution for extractive summarization.

6.1 MAIN FINDINGS

Our research demonstrates that hierarchical architectures provide advantages for text summarization. The two-level processing structure enables effective understanding of both local sentence semantics and global document structure, and sentence-level transformer attention in a hierarchical transformer is shown to be effective to rank sentences for unsupervised extractive summarization.

The experimental results on the CNN/DailyMail dataset show that our approach is highly competitive, achieving high ROUGE scores. A key finding from our ablation studies is the critical role of redundancy control. Our systematic evaluation revealed that using a PMI-based redundancy measure provides a substantial performance improvement, improving the ROUGE-1 score by 5.65 points compared to using STAS criteria alone. The optimal hyperparameter configuration, derived from a grid search, assigned zero weight to the PMI-based relevance cri-

terion, suggesting that the STAS criteria already capture document relevance effectively, while our primary contribution comes from improved redundancy management.

Component-wise analysis further demonstrates that combining multiple ranking criteria yields superior performance. The effectiveness of PMI-based measures validates the application of information-theoretic principles to summarization, showing that statistical dependencies between tokens provide more reliable redundancy signals than simpler n-gram or cosine similarity measures. Our results confirm that unsupervised approaches can achieve performance competitive with state-of-the-art supervised methods while offering significant practical advantages, such as eliminating the need for dataset-specific training.

6.2 LIMITATIONS

Despite the competitive performance, a considerable gap remains between our method and the oracle performance, with a difference of approximately 13 ROUGE-1 points. This indicates significant room for improvement in unsupervised extractive summarization. The hierarchical processing and our sentence scoring approach, while effective, requires considerable computational resources, particularly for longer documents.

Our evaluation focuses primarily on the CNN/DailyMail news dataset. While this is a standard benchmark, its specific characteristics, such as a strong lead bias, may favor our approach. The generalizability of our findings to other domains and document types, such as scientific literature or longer documents, requires further validation. Finally, while our redundancy control addresses content overlap at the token level, it may not fully capture deeper semantic coherence issues, which can affect the readability and logical flow of the generated summary.

6.3 FUTURE WORK

Future research can explore more complex hierarchical designs that can capture paragraph, section, and document structures simultaneously. Also other ranking criteria should be investigated to further enhance redundancy control and relevance assessment.

To test how well hierarchical approaches work in different domains, future research should evaluate these methods on various types of documents and datasets. This includes testing on long documents such as research papers from arXiv. This evaluation is important since models with these architectures are specifically designed to handle the challenges associated with

processing long documents, such as computational inefficiency, loss of structural information, and difficulty in capturing both local and global context.

Finally, combining unsupervised methods with limited supervised fine-tuning could leverage the strengths of both approaches. Semi-supervised learning techniques could help adapt models to specific domains while still benefiting from the generalizability of unsupervised training.

References

- [1] M. Azam, S. Khalid, S. Almutairi, H. A. Khattak, A. Namoun, A. Ali, and H. S. M. Bilal, “Current trends and advances in extractive text summarization: A comprehensive review,” *IEEE Access*, vol. 12, pp. 101 805–101 830, 2024.
- [2] W. S. El-Kassas, C. R. Salama, A. A. Rafea, and H. K. Mohamed, “Automatic text summarization: A comprehensive survey,” *Expert Systems with Applications*, vol. 166, p. 114037, 2021.
- [3] X. Zhang, F. Wei, and M. Zhou, “Hibert: Document level pre-training of hierarchical bidirectional transformers for document summarization,” in *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics*, 2019, pp. 5059–5069.
- [4] R. Mihalcea and P. Tarau, “Textrank: Bringing order into text,” in *Proceedings of the 2004 Conference on Empirical Methods in Natural Language Processing*, 2004, pp. 404–411.
- [5] J. L. Elman, “Finding structure in time,” *Cognitive Science*, vol. 14, no. 2, pp. 179–211, 1990.
- [6] S. Hochreiter and J. Schmidhuber, “Long short-term memory,” *Neural Computation*, vol. 9, no. 8, pp. 1735–1780, 1997.
- [7] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin, “Attention is all you need,” in *Advances in Neural Information Processing Systems*, vol. 30, 2017, pp. 5998–6008.
- [8] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, “Bert: Pre-training of deep bidirectional transformers for language understanding,” in *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*, 2019, pp. 4171–4186.

- [9] T. B. Brown, B. Mann, N. Ryder, M. Subbiah, J. Kaplan, P. Dhariwal, A. Neelakantan, P. Shyam, G. Sastry, A. Askell *et al.*, “Language models are few-shot learners,” in *Advances in Neural Information Processing Systems*, vol. 33, 2020, pp. 1877–1901.
- [10] S. Xu, X. Zhang, Y. Wu, F. Wei, and M. Zhou, “Unsupervised extractive summarization by pre-training hierarchical transformers,” *arXiv preprint arXiv:2010.08242*, 2020.
- [11] K. M. Hermann, T. Kocisky, E. Grefenstette, L. Espeholt, W. Kay, M. Suleyman, and P. Blunsom, “Teaching machines to read and comprehend,” in *Advances in Neural Information Processing Systems*, vol. 28, 2015.
- [12] M. Gambhir and V. Gupta, “A survey on abstractive text summarization,” in *International Conference on Computer, Communications and Electronics (Comptelix)*. IEEE, 2017, pp. 1–7.
- [13] D. R. Radev, E. Hovy, and K. McKeown, “Introduction to the special issue on summarization,” *Computational Linguistics*, vol. 28, no. 4, pp. 399–408, 2002.
- [14] A. Nenkova and K. McKeown, “A survey of text summarization techniques,” in *Mining Text Data*. Springer, 2012, pp. 43–76.
- [15] I. Mani, *Automatic summarization*. John Benjamins Publishing, 2001, vol. 3.
- [16] X. Wan, “Cross-language document summarization based on machine translation quality prediction,” in *Proceedings of the 48th Annual Meeting of the Association for Computational Linguistics*, 2010, pp. 917–926.
- [17] C.-Y. Lin, “Rouge: A package for automatic evaluation of summaries,” in *Text Summarization Branches Out: Proceedings of the ACL-04 Workshop*, 2004, pp. 74–81.
- [18] K. Papineni, S. Roukos, T. Ward, and W.-J. Zhu, “Bleu: a method for automatic evaluation of machine translation,” in *Proceedings of the 40th Annual Meeting of the Association for Computational Linguistics*, 2002, pp. 311–318.
- [19] T. Zhang, V. Kishore, F. Wu, K. Q. Weinberger, and Y. Artzi, “Bertscore: Evaluating text generation with bert,” in *International Conference on Learning Representations*, 2020.
- [20] K. Sparck Jones, “A statistical interpretation of term specificity and its application in retrieval,” *Journal of Documentation*, vol. 28, no. 1, pp. 11–21, 1972.

- [21] H. P. Luhn, "The automatic creation of literature abstracts," *IBM Journal of Research and Development*, vol. 2, no. 2, pp. 159–165, 1958.
- [22] J. Carbonell and J. Goldstein, "The use of mmr, diversity-based reranking for reordering documents and producing summaries," in *Proceedings of the 21st Annual International ACM SIGIR Conference on Research and Development in Information Retrieval*, 1998, pp. 335–336.
- [23] G. Erkan and D. R. Radev, "Lexrank: Graph-based lexical centrality as salience in text summarization," *Journal of Artificial Intelligence Research*, vol. 22, pp. 457–479, 2004.
- [24] Y. Bengio, P. Simard, and P. Frasconi, "Learning long-term dependencies with gradient descent is difficult," *IEEE Transactions on Neural Networks*, vol. 5, no. 2, pp. 157–166, 1994.
- [25] K. Cho, B. van Merriënboer, C. Gulcehre, D. Bahdanau, F. Bougares, H. Schwenk, and Y. Bengio, "Learning phrase representations using rnn encoder-decoder for statistical machine translation," in *Proceedings of the 2014 Conference on Empirical Methods in Natural Language Processing*, 2014, pp. 1724–1734.
- [26] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, 2016, pp. 770–778.
- [27] J. L. Ba, J. R. Kiros, and G. E. Hinton, "Layer normalization," *arXiv preprint arXiv:1607.06450*, 2016.
- [28] R. Sennrich, B. Haddow, and A. Birch, "Neural machine translation of rare words with subword units," in *Proceedings of the 54th Annual Meeting of the Association for Computational Linguistics*, 2016, pp. 1715–1725.
- [29] H. P. Edmundson, "New methods in automatic extracting," *Journal of the ACM*, vol. 16, no. 2, pp. 264–285, 1969.
- [30] S. Deerwester, S. T. Dumais, G. W. Furnas, T. K. Landauer, and R. Harshman, "Indexing by latent semantic analysis," *Journal of the American Society for Information Science*, vol. 41, no. 6, pp. 391–407, 1990.

- [31] Y. Gong and X. Liu, “Generic text summarization using relevance measure and latent semantic analysis,” in *Proceedings of the 24th Annual International ACM SIGIR Conference on Research and Development in Information Retrieval*, 2001, pp. 19–25.
- [32] T. K. Landauer, P. W. Foltz, and D. Laham, “An introduction to latent semantic analysis,” *Discourse Processes*, vol. 25, no. 2-3, pp. 259–284, 1998.
- [33] D. R. Radev, H. Jing, and M. Budzikowska, “Centroid-based summarization of multiple documents: sentence extraction, utility-based evaluation, and user studies,” in *Proceedings of the 2000 NAACL-ANLP Workshop on Automatic Summarization*, 2000, pp. 21–30.
- [34] A. M. Rush, S. Chopra, and J. Weston, “A neural attention model for abstractive sentence summarization,” *arXiv preprint arXiv:1509.00685*, 2015.
- [35] R. Nallapati, B. Zhou, C. dos Santos, C. Gulcehre, and B. Xiang, “Abstractive text summarization using sequence-to-sequence rnns and beyond,” *arXiv preprint arXiv:1602.06023*, 2016.
- [36] J. Cheng and M. Lapata, “Neural summarization by extracting sentences and words,” *arXiv preprint arXiv:1603.07252*, 2016.
- [37] R. Nallapati, F. Zhai, and B. Zhou, “Summarunner: A recurrent neural network based sequence model for extractive summarization of documents,” in *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 31, no. 1, 2017, pp. 3075–3081.
- [38] Y. Liu, “Fine-tune bert for extractive summarization,” *arXiv preprint arXiv:1903.10318*, 2019.
- [39] H. Zheng and M. Lapata, “Pacsum: Position-aware neural extractive summarization with centrality,” in *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics*, 2019, pp. 347–357.
- [40] T. Gokhan, P. Smith, and M. Lee, “Gusum: Graph-based unsupervised summarization using sentence features scoring and Sentence-BERT,” in *Proceedings of TextGraphs-16: Graph-based Methods for Natural Language Processing*. Gyeongju, Republic of Korea: Association for Computational Linguistics, 2022, pp. 44–53. [Online]. Available: <https://aclanthology.org/2022.textgraphs-1.5>

- [41] V. Padmakumar and H. He, “Unsupervised extractive summarization using pointwise mutual information,” in *Proceedings of the 16th Conference of the European Chapter of the Association for Computational Linguistics: Student Research Workshop*, 2021, pp. 133–142.
- [42] N. Joshi, T. Goncalves, and K. Roberts, “Ranksum: Rank fusion for multi-document summarization,” in *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, 2023, pp. 14 986–15 002.
- [43] Q. Ruan, M. Ostendorff, and G. Rehm, “Histruct+: Improving extractive text summarization with hierarchical structure information,” *arXiv preprint arXiv:2203.09629*, 2022.
- [44] I. Beltagy, M. E. Peters, and A. Cohan, “Longformer: The long-document transformer,” *arXiv preprint arXiv:2004.05150*, 2020.
- [45] M. Zaheer, G. Guruganesh, K. A. Dubey, J. Ainslie, C. Alberti, S. Ontanon, P. Pham, A. Ravula, Q. Wang, L. Yang *et al.*, “Big bird: Transformers for longer sequences,” in *Advances in Neural Information Processing Systems*, vol. 33, 2020, pp. 17 283–17 297.
- [46] M. Ott, S. Edunov, A. Baevski, A. Fan, S. Gross, N. Ng, D. Grangier, and M. Auli, “fairseq: A fast, extensible toolkit for sequence modeling,” in *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Demonstrations*, 2019, pp. 48–53, <https://fairseq.readthedocs.io/>.
- [47] X. Zhang, “Hibert: Document level pre-training of hierarchical bidirectional transformers for document summarization,” Project page, 2019, <https://xingxingzhang.github.io/hibert.html>.
- [48] S. Xu, “Stas: Unsupervised extractive summarization by pre-training hierarchical transformers,” GitHub repository, 2020, <https://github.com/xsstory/STAS>.
- [49] Facebook AI Research, “Fairseq documentation,” Online, 2024, <https://fairseq.readthedocs.io/en/latest/>.
- [50] A. See, P. J. Liu, and C. D. Manning, “Get to the point: Summarization with pointer-generator networks,” in *Proceedings of the 55th Annual Meeting of the Association for Computational Linguistics*, vol. 1, 2017, pp. 1073–1083.

[51] Lightning AI, “Lightning ai,” <https://lightning.ai/>.